



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola d'Enginyeria de Telecomunicació
i Aeroespacial de Castelldefels

Mobilitat, Xarxes i Serveis

Wireless Multihop Network Project

Titulació: Professor Carles Gomez Montenegro

Assignatura: Mobilitat, Xarxes i Serveis

Authors:

Daniel Ceron Espinosa

Sayna Kiani

Ali Mehrabikouchehbiouk

Alejandro Valle Gonzalez

Data: 23/05/2026

Index

Session 1	1
Session 2	12
Session 3	26
Conclusion	42

SESSION 1:

Inifile: A file that gives you the settings for all the parameters/variables used in the simulation.

Nedfile: Modules supported in the simulation.

1. Which protocol layer is being considered in the simulation?

Application layer, transport layer UDP, network/routing layer AODV/IP, MAC layer, and physical layer.

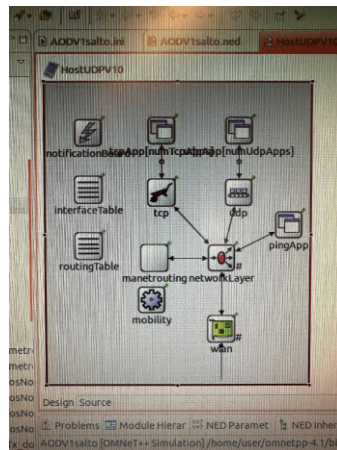


Figure 1: Host ned file

We send packets over the WiFi interface.

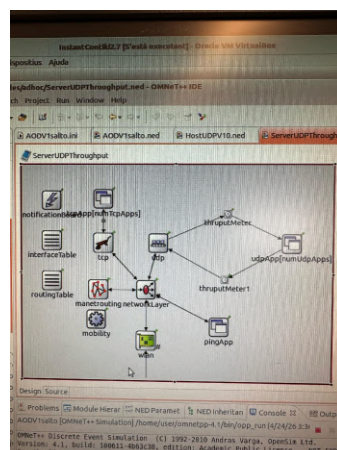


Figure 2: Server ned file

In the receiver we have a program that measures how many bits per second are being received.

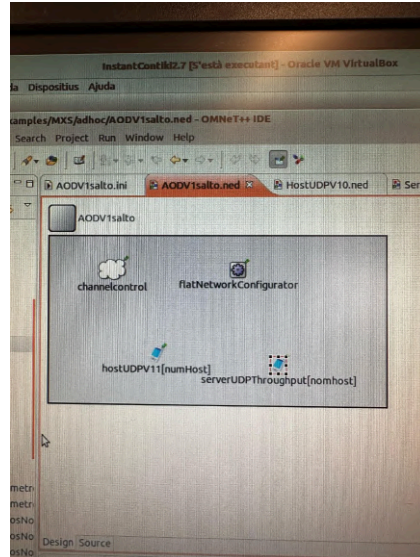


Figure 3: Scenario

There is a parameter that needs to be changed in the initial file, for the 4 hops scenario.

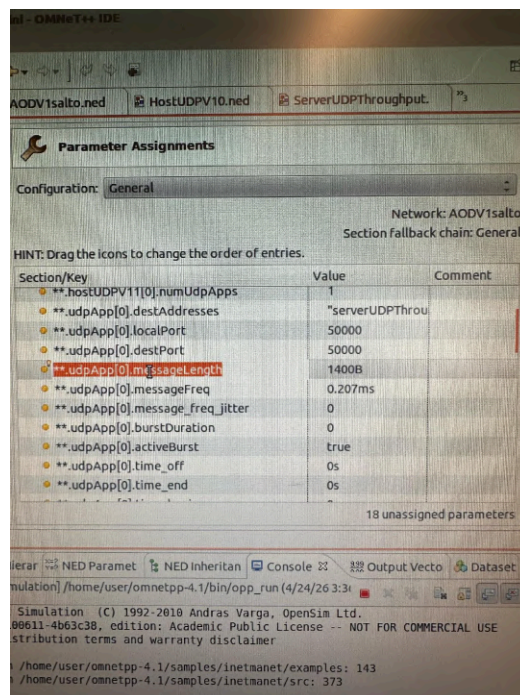


Figure 4: Scenario

The application layer packet that we want to send is 1400B. Then we will add the WiFi header, the UDP header. Then, we will send it.

The `udpApp[0].messageFreq` is 0,207ms. This is how often you are attempting to transmit from the application.

To sum up, what the application is trying to do is to create a flow:

$$\text{bps} = 1400 \times 8 / 0,000207 = 54.106.280,193 \text{ bps}$$

This value is approximately 54 Mbps. This is the traffic generated by the application.

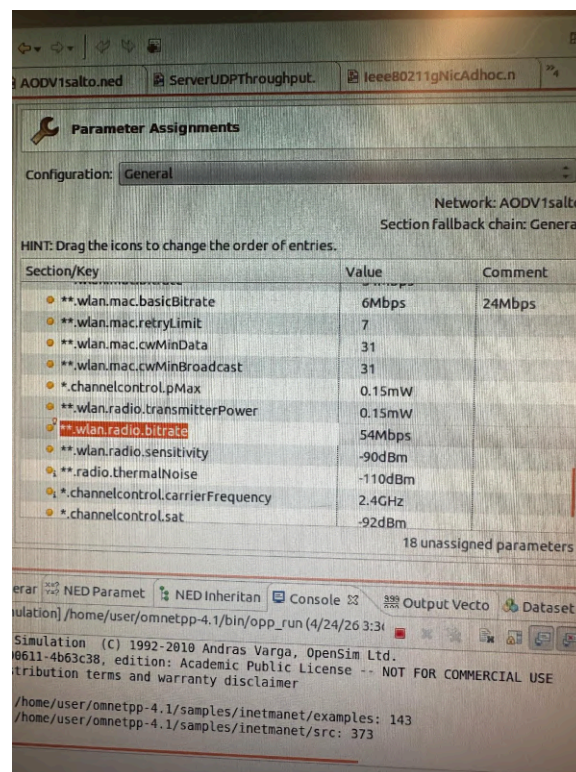


Figure 5: Setting for the physical layer bit rate

Will the client node configuration allow to evaluate the maximum bandwidth that can be obtained in a communication between two devices?

The traffic generated by the application is the same as the physical bitrate configured. However, in the real world, the throughput will be lower because we have protocol overheads such as WiFi, UDP/IP headers, MAC access time, ACK's, etc. Yes, we will be

able to evaluate the maximum bandwidth but we need to take into account what we said before.

The application generates a flow of 54,1 Mbps. It is greater than the physical layer bit rate. The sender is sending frames all the time at the highest possible rate. We have a frame, and then, another is ready to be transmitted. So what we get is maximum end-to-end throughput.

3.- Execute the scenario simulation. This is the obtained throughput?

We simulate the scenario and we obtain the following graph for the throughput. What we get is approximately 23 Mbps. We do not have interference as we are the only ones sending packets. We are alone (there is no external competing flow, but the throughput is still lower because of IEEE 802.11 protocol overhead).

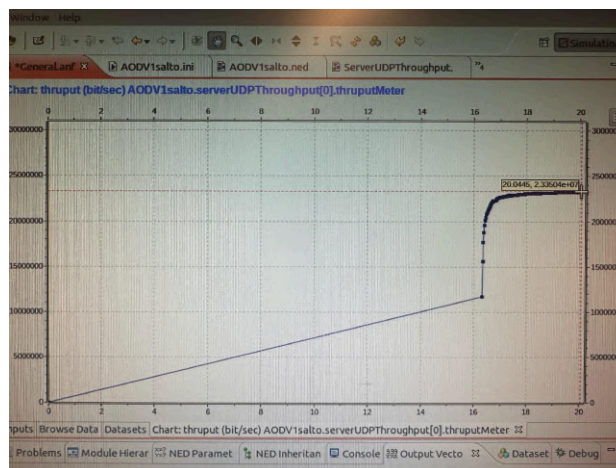


Figure 6: AODV1salto

Why do we get a much lower throughput?

Because of the protocol overheads (MAC layer protocol). We lose some time in other operations and that's why we don't get the theoretical 54 Mbps.

In the one-hop case, even though the sender is configured to generate traffic at a rate slightly higher than the physical layer bit rate and its radio always attempts to transmit at the maximum possible rate, the actual throughput observed (around 23 Mbps) is significantly lower due to several factors. First, there is protocol overhead: each transmitted frame includes multiple headers (MAC, PHY, etc.) in addition to the payload, and these headers consume transmission time without contributing to useful application-layer data. Propagation delay is negligible because the distance is short, so it is not a limiting factor. However, the key limitation comes from the medium access mechanism used by Wi-Fi, specifically CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance). The sender cannot transmit immediately; it must first listen to the medium to ensure it is idle, which introduces waiting time during which no data is sent. Even in the absence of other transmitters, this listening period is mandatory. Additionally, certain parts of the transmission, such as headers, are sent at a lower bit rate for reliability, further reducing efficiency. After sending the payload, the sender cannot immediately proceed with the next frame—it must wait for an acknowledgment (ACK) from the receiver to confirm successful delivery. Only after receiving the ACK can the transmission cycle restart. Altogether, these MAC-layer mechanisms (channel sensing, waiting periods, lower-rate headers, and ACKs) consume a substantial portion of time, leading to roughly 50% of the time being effectively “wasted” on coordination rather than actual data transmission.

Next, execute the simulation for the rest of single flow, string topology scenarios.

In the **two-hop case**, the available bandwidth is no longer constant and effectively decreases compared to the one-hop scenario because the communication is wireless and all nodes share the same radio medium, leading to interference among them; unlike a wired multi-hop network where links are isolated, here transmissions on one link affect the others. For a path $A \rightarrow B \rightarrow C$, node B must first receive packets from A and then forward

them to C, but since it typically has a single radio interface, it cannot transmit and receive at the same time (half-duplex constraint). Moreover, nodes A and B share the same channel, so they cannot transmit simultaneously without causing collisions, meaning that

only one of the two links (A–B or B–C) can be active at any given time. As a result, each link is effectively used only about 50% of the time, since transmissions must alternate between the two hops. Even if each individual link could achieve a certain capacity, the end-to-end throughput is limited by this time-sharing, effectively dividing the capacity by two. Consequently, if the one-hop throughput was around 23 Mbps, the two-hop end-to-end throughput drops to roughly half, about 12 Mbps, due to contention, interference, and the half-duplex nature of wireless communication.

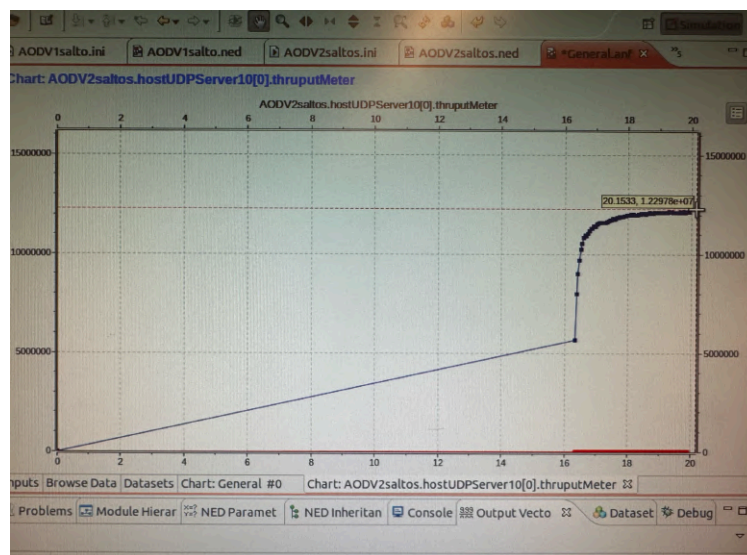


Figure 7: AODV2salto

In the **three-hop case** ($A \rightarrow B \rightarrow C \rightarrow D$), the situation becomes even more constrained due to wireless interference and the hidden node problem: although in theory nodes like A and C might try to transmit at the same time to improve efficiency, in practice this causes issues because WiFi radios use omnidirectional antennas, meaning that when C transmits to D it also broadcasts in all directions, including toward B; as a result, node B may simultaneously receive signals from both A and C, leading to collisions or interference, which is a classic example of the hidden node problem. Because of this, the network

must avoid overlapping transmissions along the path to ensure successful delivery, effectively forcing transmissions on links A-B, B-C, and C-D to occur at different times. This means that, in practice, only one link can be usefully active at a time along the path, so each link gets roughly one third of the channel usage. Consequently, even if the nominal one-hop throughput is about 23 Mbps, the end-to-end throughput over three hops is reduced by a factor of three, resulting in approximately 7 Mbps, due to interference, medium contention, and the need to serialize transmissions across the hops.

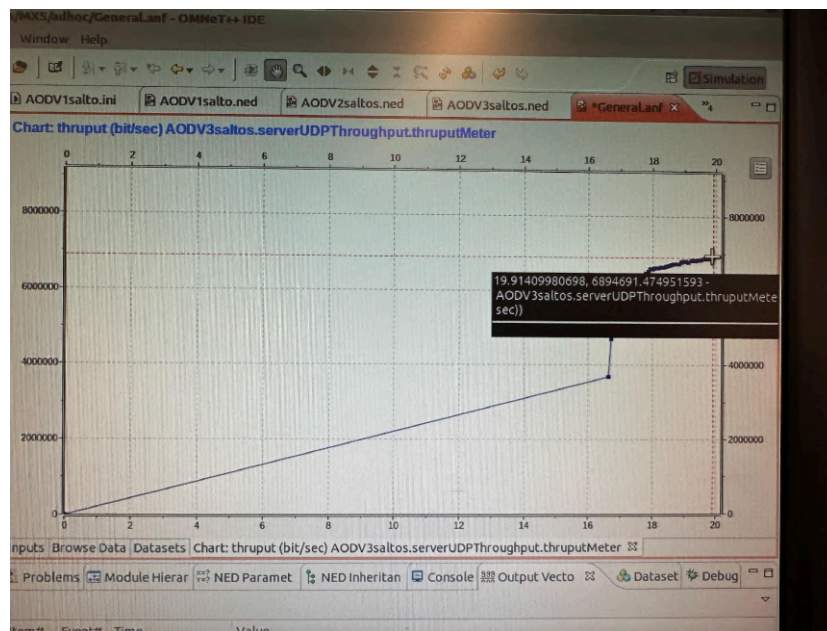


Figure 8: AODV3salto

In the **four-hop case** ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$), the end-to-end throughput degrades even further due to both contention and interference effects: not only can adjacent nodes not transmit simultaneously, but even non-adjacent transmissions become problematic because of the difference between transmission range and interference range, where the latter is larger; this means that when a node like D transmits, its signal can still propagate far enough to interfere with nodes such as C or even E, even if they are not the intended receivers, causing collisions or corrupted receptions. As a result, transmissions along the path cannot be spatially reused effectively, and the network must serialize communication across all four links to avoid interference. This prevents simultaneous transmissions between nodes like A, B, C, and D, even if they are not direct neighbors. Consequently, only one link among A-B, B-C, C-D, or D-E can be actively transmitting useful data at

any given time. Therefore, each link effectively gets only one quarter of the channel usage, and the end-to-end throughput is reduced by a factor of four compared to the one-hop case, meaning that if the base throughput was around 23 Mbps, it drops to roughly 5–6 Mbps due to cumulative interference, medium access constraints, and the expanded interference range in multi-hop wireless networks.

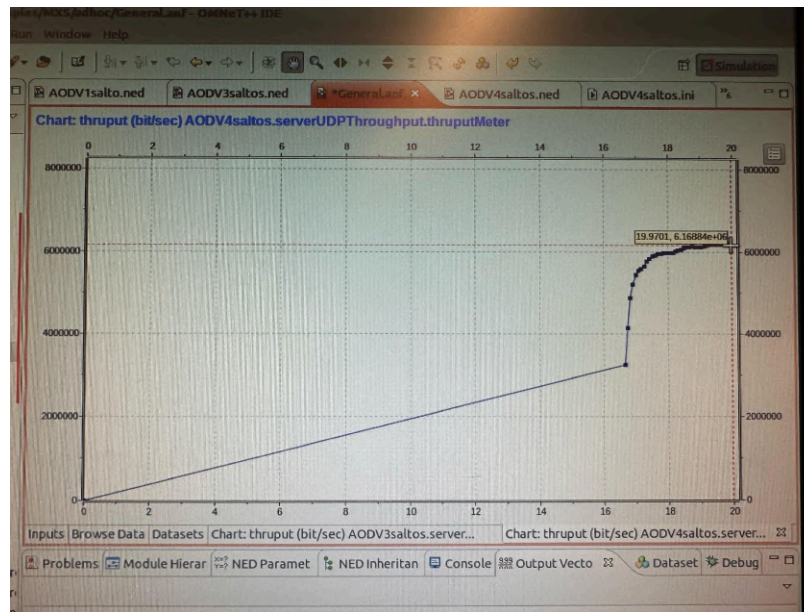


Figure 9: AODV4salto

What if we tried to add a fifth hope?

In the five-hop case ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$), the throughput does not necessarily keep decreasing simply by dividing the one-hop capacity by 5, because with a longer chain there may finally be enough distance between some links to allow spatial reuse. In the four-hop case, only one useful transmission could occur at a time because the interference range was large enough to affect the other links, so the end-to-end capacity was roughly the one-hop capacity divided by 4. However, with five hops, links that are far enough apart, for example $A \rightarrow B$ and $E \rightarrow F$, may be able to transmit at the same time without interfering with each other, depending on the interference range shown in the image. This means the network can reuse the same wireless channel in different parts of the path at the same time. Therefore, the capacity does not simply become $23 \text{ Mbps} / 5$; instead, it tends to remain around the same order as the four-hop case, roughly $23 \text{ Mbps} / 4$, so about 5–6 Mbps, because the limiting factor is not the total number of hops

anymore but how many links must be kept silent around each active transmission due to carrier sensing, half-duplex radios, omnidirectional antennas, and interference range.

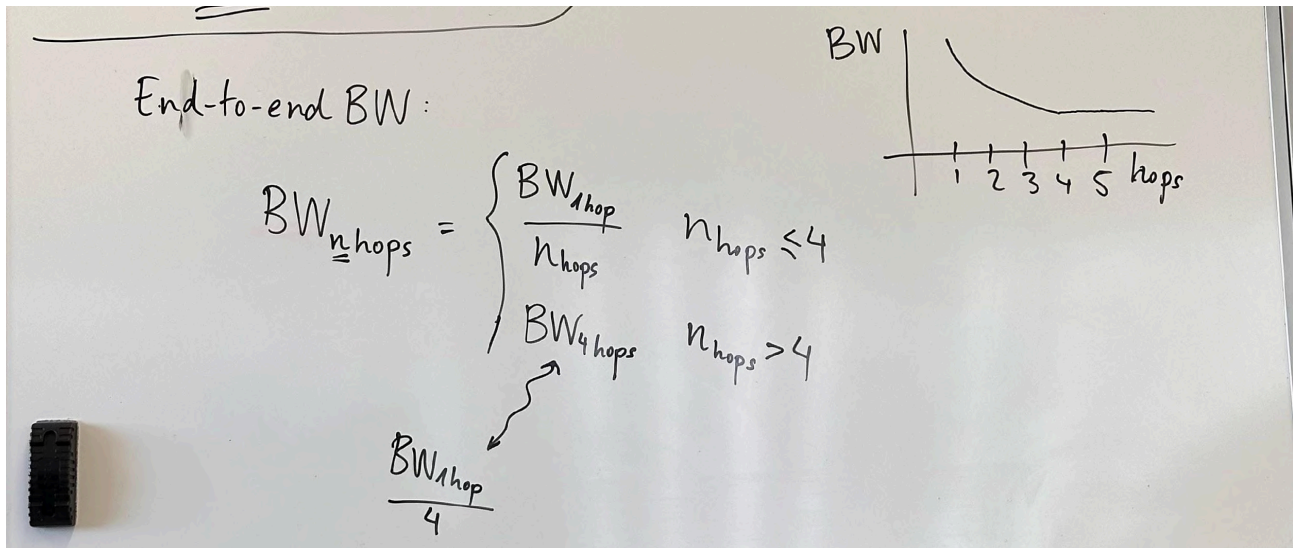


Figure 10: Relationship between the number of hops and the available capacity

Two flows via different topology paths: In the case of two simultaneous flows over wireless multi-hop paths, the behavior you observe in the graph is mainly due to interference and fair sharing of the medium: when only one flow is active, it can exploit the channel as much as possible and achieves a higher throughput (similar to the single-flow multi-hop case, e.g., around the value corresponding to 3 hops). However, when the second flow becomes active, both flows start competing for the same wireless medium under the same conditions (same contention rules, same transmission rate, same CSMA/CA mechanism), so the available capacity must be shared between them. If the two paths are close enough, they interfere with each other, meaning their transmissions cannot occur simultaneously in many parts of the network, which effectively couples them into a single shared resource. As a result, the throughput of each flow drops to approximately half of what a single flow would achieve, reflecting fair access to the medium. This is why in the graph you see that once both flows are active (around $t \approx 17$ s), the higher-throughput curve decreases and stabilizes, while the lower one increases, and both converge to values that are roughly half of the single-flow capacity. If the flows

were far enough apart (no interference), they could each achieve close to the single-flow throughput independently, but in this scenario they are close, so the result matches the multi-hop interference behavior (similar to the 3-hop case) with the total capacity being shared evenly between the two flows.

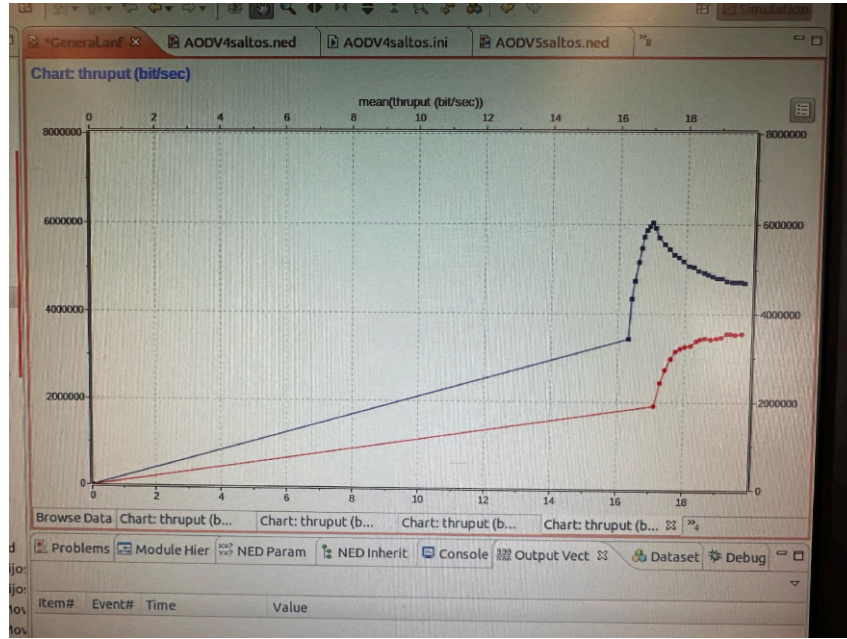


Figure 11: 2 flows close to each other

Extending the scenarios: In this extended scenario, the positions of the nodes were modified in order to guarantee end-to-end connectivity across the entire deployment area while maintaining a reasonable number of relay nodes. The main objective was to ensure that there was always a neighboring node available for forwarding packets, allowing both communication flows to reach their respective destinations successfully. During the design process, the coordinates of the nodes were adjusted carefully, taking into account that the coordinate (0,0) is considered outside the playground and therefore cannot be used for node placement. Unlike the previous string topologies, this scenario represents a more realistic deployment in which multiple communication paths coexist within the same network. The obtained results show an end-to-end throughput of approximately 2 Mbps at each receiver, confirming that both destinations are able to receive data correctly. The reduction in throughput compared to the previous scenarios is mainly caused by the increased number of transmissions sharing the wireless medium and by the interference

generated between nearby links. Although the throughput is lower than in the single-flow scenarios, connectivity is successfully maintained across the whole network, which is the primary requirement of the rescue-team application. From a service perspective, a throughput of around 2 Mbps is generally sufficient for voice communications and the transmission of reports, maps, and sensor information, while more demanding applications such as high-quality video streaming may experience limitations. Therefore, the proposed deployment represents a compromise between coverage, connectivity, and available bandwidth, providing acceptable performance for emergency communication services while keeping the number of deployed devices relatively low.

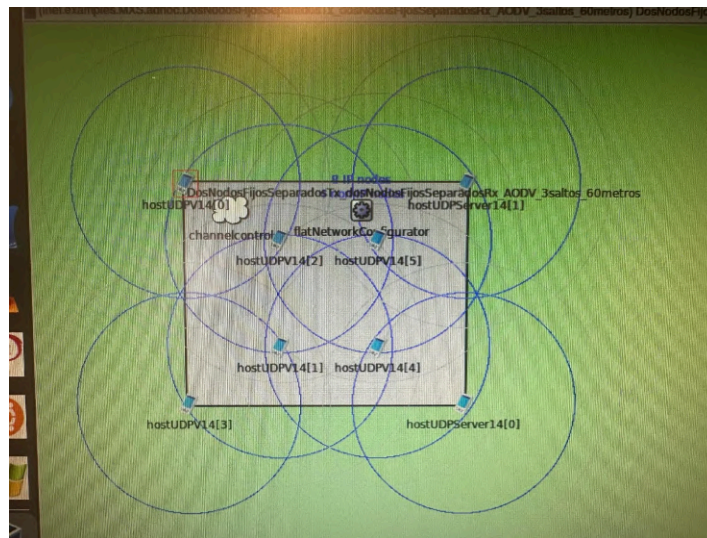


Figure 12: Extended network topology with modified node positions.

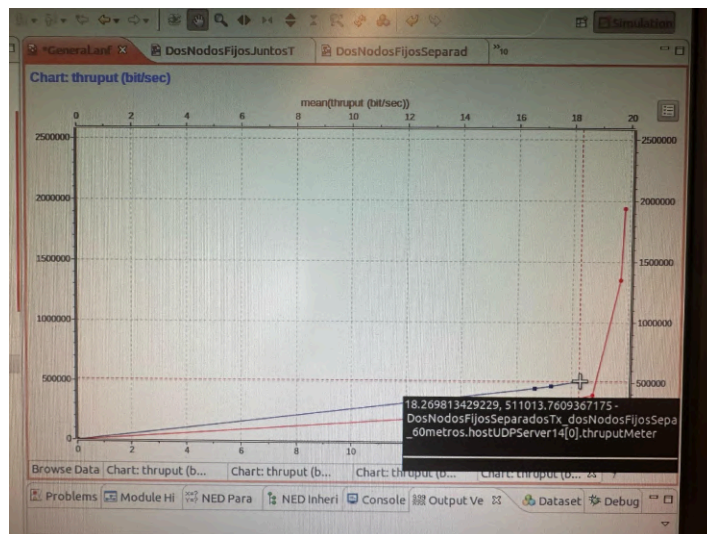


Figure 13: Throughput obtained by Flow 1 in the extended scenario.

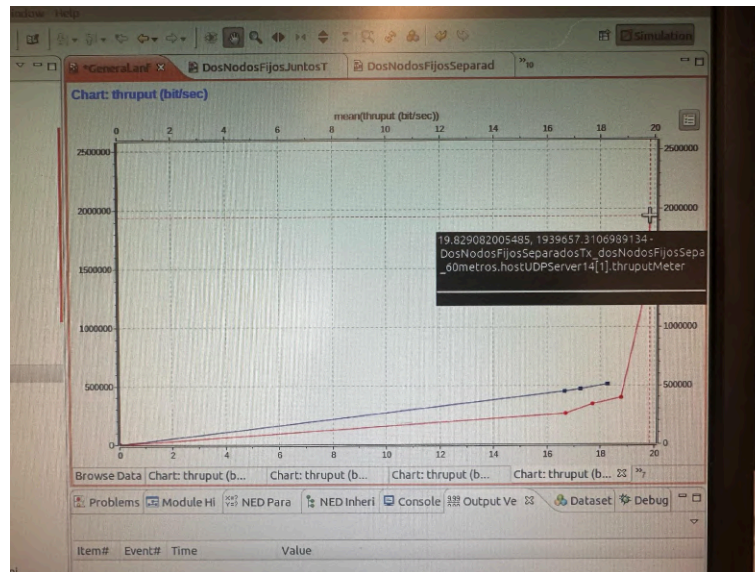


Figure 14: Throughput obtained by Flow 2 in the extended scenario.

SESSION 2:

In this session, the goal is to evaluate routing protocols in a real wireless multihop network. A routing protocol is responsible for establishing end-to-end paths and detecting failures, allowing the network to switch to an alternative route when a path is broken.

Two main approaches are considered: proactive routing (OLSR), which maintains routes continuously, and reactive routing (AODV), which discovers routes only when needed. The project involves experimenting with both protocols and selecting one based on performance and justification.

The evaluation is based on three key parameters: end-to-end latency, end-to-end bandwidth, and route change latency (RCL), which measures how fast the protocol reacts to failures.

The first scenario is a chain topology, where nodes are arranged linearly to analyze performance as the number of hops increases. Initial measurements focus on latency, using tools like ping to observe how delay evolves in the network.

Introduce the code given.

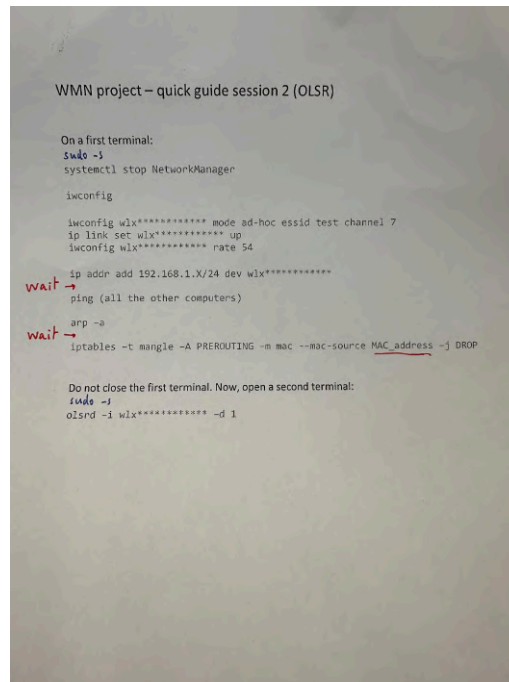


Figure 15: Code given

First, we run the `iwconfig` command to retrieve information about the wireless interface, including the Wi-Fi card's MAC address, which is `wlan0`.

Next, we continue executing additional commands. The `arp` command provides information about the MAC addresses of other devices on the network and helps verify that connectivity is established within a single hop.

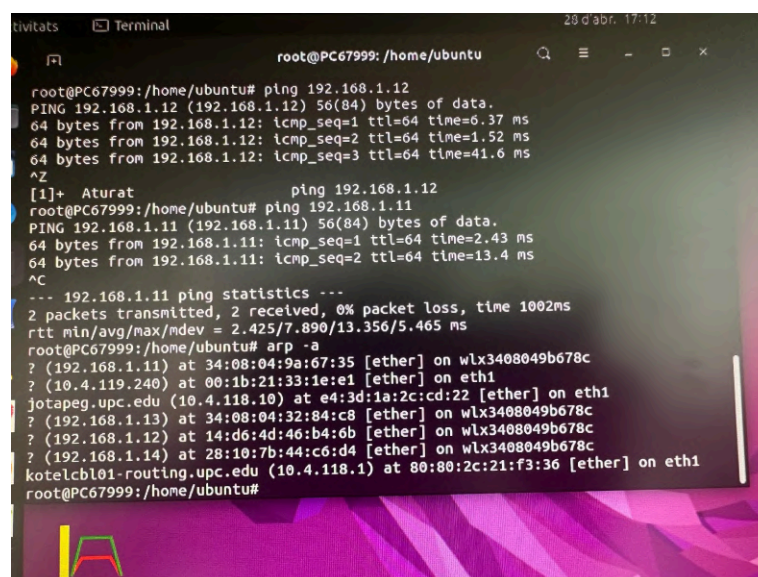


Figure 16: Information given by the `arp` command

In a multihop network, nodes are not necessarily direct neighbors of each other. For example, nodes 11 and 13 should not be one-hop neighbors, meaning they should not be able to communicate directly. Instead, communication between them should occur through intermediate nodes, allowing us to evaluate true multihop behavior.

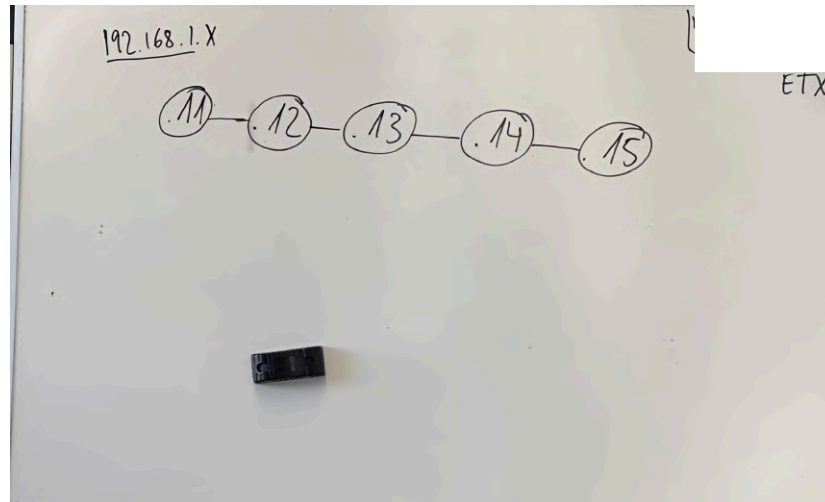


Figure 17: Scenario

A natural way to achieve this would be to physically separate the computers and place them in different locations so that only certain nodes are within transmission range of each other. However, this approach introduces an important issue: the wireless environment is highly dynamic. Factors such as people moving around or changes in the surroundings can affect link quality.

As a result, even if we initially design a topology with, for example, three hops between two nodes, this may not remain stable over time. The routing protocol continuously evaluates link conditions and may change routes dynamically. Therefore, the number of hops between nodes is not fixed and can vary depending on the current network conditions, making experiments less controlled and harder to reproduce.

Instead of relying on a fully real wireless setup, the best approach is to emulate the desired network behavior while keeping all devices physically in the same place. This avoids the instability caused by environmental changes and allows for controlled and repeatable experiments.

To achieve this, we artificially enforce the network topology by filtering traffic based on MAC addresses. Specifically, each node is configured to discard all traffic coming from nodes that are not its direct neighbors. This means that communication links are selectively enabled or disabled, effectively emulating a multihop topology.

For example, node 13 can be configured to reject all traffic from nodes 11 and 15, ensuring that it only communicates with its intended neighbors. This approach allows us to precisely control which nodes are considered one-hop neighbors and guarantees a stable multihop scenario for evaluation.

```
? (192.168.1.13) at 34:08:04:32:84:c8 [ether] on wlx3408049b678c
? (192.168.1.12) at 14:d6:4d:46:b4:6b [ether] on wlx3408049b678c
? (192.168.1.14) at 28:10:7b:44:c6:d4 [ether] on wlx3408049b678c
kotelcbl01-routing.upc.edu (10.4.118.1) at 80:80:2c:21:f3:36 [ether] on eth1
root@PC67999:/home/ubuntu# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:9a:67:35 -j DROP
root@PC67999:/home/ubuntu# iptables -t mangle -A PREROUTING -m mac --mac-source 14:d6:4d:46:b4:6b -j DROP
root@PC67999:/home/ubuntu# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:32:84:c8 -j DROP
root@PC67999:/home/ubuntu#
```

Figure 18: MAC filtering rules used to force the multi-hop topology (node 15)

```
Try 'iptables -h' or 'iptables --help' for more information.
root@PC68031:/home/ubuntu# iptables -t mangle -A PREROUTING -m mac --mac-source
34:08:04:9a:67:35 -j DROP
root@PC68031:/home/ubuntu# iptables -t mangle -A PREROUTING -m mac --mac-source
14:d6:4d:46:b4:6b -j DROP
root@PC68031:/home/ubuntu#
```

Figure 19: MAC filtering rules used to force the multi-hop topology (node 14)

After configuring the wireless interface and starting OLSR with the command `olsrd -i wlx***** -d 1`, the routing protocol displays information about direct neighbors, two-hop neighbors, and link quality metrics (ETX). This information is useful to verify the discovered topology and the routes computed by OLSR.

```
root@PC67999:/home/ubuntu
*** olsr.org - 0.6.6.2-git_0000000-hash_12fa41b5362519d37bea6715ce291978
(2014-09-11 13:09:19 on toyol) ***

--- 17:27:04.060217 --- LINKS
IP address      hyst      LQ      ETX
192.168.1.14    0.000    1.000/1.000    1.000    <d>

--- 17:27:04.060232 --- NEIGHBORS
IP address      LQ      NLQ      SYN      MPR      MPRS      will
192.168.1.14    0.000    YES      YES      NO      3

--- 17:27:04.060239 --- TWO-HOP NEIGHBORS
IP addr (2-hop) IP addr (1-hop) Total cost
192.168.1.13    192.168.1.14    2.000
```

Figure 20: OLSR output showing node 14 as direct neighbor, and node 13 as a two-hop neighbor (from node 15 point of view)



Figure 21: OLSR output showing node 13 and node 15 as direct neighbors, and node 12 as a two-hop neighbor (from node 14 point of view)

In the OLSR routing protocol, each link between nodes is assigned a cost that is used to determine the best path to a destination. The protocol selects routes based on the total cost of the path, choosing the one with the lowest overall value.

In the interface shown in the image, there is a column labeled ETX (Expected Transmission Control). This metric represents the expected number of transmissions required to successfully deliver a packet over a link. It is defined as :

$$ETX = 1 / P, \text{ where } P \text{ is the probability of successful packet delivery.}$$

An ETX value close to 1 indicates a high-quality link, where packets are successfully delivered with minimal retransmissions. This is the ideal situation in stable network conditions. However, if the ETX value increases, it means that more retransmissions are needed, which indicates degradation in link quality.

Therefore, an increase in ETX is bad news: it reflects problems in the link, such as interference, noise, or unstable conditions. The routing protocol uses this information to avoid poor-quality links and select more reliable paths whenever possible.

We perform ping tests from node 15 with one hop, two hops, three hops, and so on. Unusual values appear due to the ETX metric.

```
root@PC67999:/home/ubuntu# iptables -t mangle -A PREROUTING -m mac --mac-source 192.168.1.14 -j ACCEPT
root@PC67999:/home/ubuntu# ping 192.168.1.14
PING 192.168.1.14 (192.168.1.14) 56(84) bytes of data:
64 bytes from 192.168.1.14: icmp_seq=1 ttl=64 time=0.736 ms
64 bytes from 192.168.1.14: icmp_seq=2 ttl=64 time=0.900 ms
64 bytes from 192.168.1.14: icmp_seq=3 ttl=64 time=0.903 ms
64 bytes from 192.168.1.14: icmp_seq=4 ttl=64 time=0.904 ms
64 bytes from 192.168.1.14: icmp_seq=5 ttl=64 time=0.765 ms
64 bytes from 192.168.1.14: icmp_seq=6 ttl=64 time=0.860 ms
64 bytes from 192.168.1.14: icmp_seq=7 ttl=64 time=0.738 ms
64 bytes from 192.168.1.14: icmp_seq=8 ttl=64 time=0.893 ms
64 bytes from 192.168.1.14: icmp_seq=9 ttl=64 time=0.879 ms
64 bytes from 192.168.1.14: icmp_seq=10 ttl=64 time=0.868 ms
64 bytes from 192.168.1.14: icmp_seq=11 ttl=64 time=0.880 ms
64 bytes from 192.168.1.14: icmp_seq=12 ttl=64 time=0.881 ms
64 bytes from 192.168.1.14: icmp_seq=13 ttl=64 time=0.904 ms
64 bytes from 192.168.1.14: icmp_seq=14 ttl=64 time=0.759 ms
64 bytes from 192.168.1.14: icmp_seq=15 ttl=64 time=0.834 ms
64 bytes from 192.168.1.14: icmp_seq=16 ttl=64 time=0.690 ms
64 bytes from 192.168.1.14: icmp_seq=17 ttl=64 time=0.790 ms
64 bytes from 192.168.1.14: icmp_seq=18 ttl=64 time=0.906 ms
64 bytes from 192.168.1.14: icmp_seq=19 ttl=64 time=0.904 ms
64 bytes from 192.168.1.14: icmp_seq=20 ttl=64 time=0.610 ms
64 bytes from 192.168.1.14: icmp_seq=21 ttl=64 time=0.733 ms
^C
--- 192.168.1.14 ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 20460ms
rtt min/avg/max/mdev = 0.610/0.825/0.906/0.084 ms
```

Figure 22: ping 192.168.1.14

The values are :

- Min RTT : 0.610 ms,
- Avg RTT : 0.825 ms
- Max RTT : 0.906 ms
- mDev : 0.084 ms

```
root@PC67999:/home/ubuntu# ping 192.168.1.13
PING 192.168.1.13 (192.168.1.13) 56(84) bytes of data:
64 bytes from 192.168.1.13: icmp_seq=1 ttl=63 time=1.46 ms
64 bytes from 192.168.1.13: icmp_seq=2 ttl=63 time=1.65 ms
64 bytes from 192.168.1.13: icmp_seq=3 ttl=63 time=1.33 ms
64 bytes from 192.168.1.13: icmp_seq=4 ttl=63 time=1.42 ms
64 bytes from 192.168.1.13: icmp_seq=5 ttl=63 time=1.44 ms
64 bytes from 192.168.1.13: icmp_seq=6 ttl=63 time=1.61 ms
64 bytes from 192.168.1.13: icmp_seq=7 ttl=63 time=5.78 ms
64 bytes from 192.168.1.13: icmp_seq=8 ttl=63 time=9.70 ms
64 bytes from 192.168.1.13: icmp_seq=9 ttl=63 time=2.44 ms
64 bytes from 192.168.1.13: icmp_seq=10 ttl=63 time=1.86 ms
64 bytes from 192.168.1.13: icmp_seq=11 ttl=63 time=53.5 ms
64 bytes from 192.168.1.13: icmp_seq=14 ttl=63 time=1.63 ms
64 bytes from 192.168.1.13: icmp_seq=15 ttl=63 time=20.4 ms
64 bytes from 192.168.1.13: icmp_seq=17 ttl=63 time=17.9 ms
64 bytes from 192.168.1.13: icmp_seq=18 ttl=63 time=5.43 ms
64 bytes from 192.168.1.13: icmp_seq=44 ttl=63 time=43.7 ms
64 bytes from 192.168.1.13: icmp_seq=45 ttl=63 time=10.6 ms
64 bytes from 192.168.1.13: icmp_seq=47 ttl=63 time=11.2 ms
64 bytes from 192.168.1.13: icmp_seq=48 ttl=63 time=26.1 ms
64 bytes from 192.168.1.13: icmp_seq=51 ttl=63 time=2.39 ms
^C
--- 192.168.1.13 ping statistics ---
51 packets transmitted, 20 received, 60.7843% packet loss, time 50686ms
rtt min/avg/max/mdev = 1.333/11.073/53.488/14.423 ms
root@PC67999:/home/ubuntu# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data:
```

Figure 23: ping 192.168.1.13

The values are :

- Min RTT : 1.333 ms,
- Avg RTT : 11.073 ms
- Max RTT : 53.488 ms
- mDev : 14.423 ms

```
64 bytes from 192.168.1.13: icmp_seq=51 ttl=63 time=2.39 ms
^C
--- 192.168.1.13 ping statistics ---
51 packets transmitted, 20 received, 60.7843% packet loss, time 50686ms
rtt min/avg/max/mdev = 1.333/11.073/53.488/14.423 ms
root@PC67999:/home/ubuntu# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data:
64 bytes from 192.168.1.12: icmp_seq=1 ttl=62 time=2.32 ms
64 bytes from 192.168.1.12: icmp_seq=2 ttl=62 time=2.31 ms
64 bytes from 192.168.1.12: icmp_seq=3 ttl=62 time=2.29 ms
64 bytes from 192.168.1.12: icmp_seq=4 ttl=62 time=2.49 ms
64 bytes from 192.168.1.12: icmp_seq=5 ttl=62 time=2.40 ms
64 bytes from 192.168.1.12: icmp_seq=6 ttl=62 time=2.40 ms
64 bytes from 192.168.1.12: icmp_seq=7 ttl=62 time=2.33 ms
64 bytes from 192.168.1.12: icmp_seq=8 ttl=62 time=2.34 ms
64 bytes from 192.168.1.12: icmp_seq=9 ttl=62 time=1.98 ms
64 bytes from 192.168.1.12: icmp_seq=10 ttl=62 time=2.46 ms
64 bytes from 192.168.1.12: icmp_seq=11 ttl=62 time=2.22 ms
64 bytes from 192.168.1.12: icmp_seq=12 ttl=62 time=5.33 ms
64 bytes from 192.168.1.12: icmp_seq=13 ttl=62 time=2.21 ms
64 bytes from 192.168.1.12: icmp_seq=14 ttl=62 time=2.39 ms
64 bytes from 192.168.1.12: icmp_seq=15 ttl=62 time=34.8 ms
64 bytes from 192.168.1.12: icmp_seq=16 ttl=62 time=3.00 ms
64 bytes from 192.168.1.12: icmp_seq=17 ttl=62 time=2.30 ms
64 bytes from 192.168.1.12: icmp_seq=18 ttl=62 time=3.51 ms
64 bytes from 192.168.1.12: icmp_seq=19 ttl=62 time=2.60 ms
64 bytes from 192.168.1.12: icmp_seq=20 ttl=62 time=2.99 ms
64 bytes from 192.168.1.12: icmp_seq=21 ttl=62 time=2.17 ms
^C
--- 192.168.1.12 ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 20034ms
rtt min/avg/max/mdev = 1.975/4.133/34.760/6.883 ms
root@PC67999:/home/ubuntu#
```

Figure 24: ping 192.168.1.12

The values are :

- Min RTT : 1.975 ms,
- Avg RTT : 4.133 ms
- Max RTT : 34.760 ms
- mDev : 6.883 ms

```
--- 192.168.1.12 ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 20034ms
rtt min/avg/max/mdev = 1.975/4.133/34.760/6.883 ms
root@PC67999:/home/ubuntu# ping 192.168.1.11
PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data:
64 bytes from 192.168.1.11: icmp_seq=1 ttl=61 time=3.26 ms
64 bytes from 192.168.1.11: icmp_seq=2 ttl=61 time=3.42 ms
64 bytes from 192.168.1.11: icmp_seq=3 ttl=61 time=3.26 ms
64 bytes from 192.168.1.11: icmp_seq=4 ttl=61 time=3.12 ms
64 bytes from 192.168.1.11: icmp_seq=5 ttl=61 time=3.13 ms
64 bytes from 192.168.1.11: icmp_seq=6 ttl=61 time=2.84 ms
64 bytes from 192.168.1.11: icmp_seq=7 ttl=61 time=3.21 ms
64 bytes from 192.168.1.11: icmp_seq=8 ttl=61 time=3.21 ms
64 bytes from 192.168.1.11: icmp_seq=9 ttl=61 time=3.72 ms
64 bytes from 192.168.1.11: icmp_seq=10 ttl=61 time=3.31 ms
64 bytes from 192.168.1.11: icmp_seq=11 ttl=61 time=3.03 ms
64 bytes from 192.168.1.11: icmp_seq=12 ttl=61 time=3.57 ms
64 bytes from 192.168.1.11: icmp_seq=16 ttl=61 time=3.08 ms
64 bytes from 192.168.1.11: icmp_seq=17 ttl=61 time=3.79 ms
64 bytes from 192.168.1.11: icmp_seq=18 ttl=61 time=5.72 ms
64 bytes from 192.168.1.11: icmp_seq=19 ttl=61 time=3.44 ms
64 bytes from 192.168.1.11: icmp_seq=20 ttl=61 time=3.33 ms
64 bytes from 192.168.1.11: icmp_seq=21 ttl=61 time=6.87 ms
64 bytes from 192.168.1.11: icmp_seq=22 ttl=61 time=3.15 ms
^C
--- 192.168.1.11 ping statistics ---
22 packets transmitted, 19 received, 13.6364% packet loss, time 21094ms
rtt min/avg/max/mdev = 2.835/3.601/6.869/0.968 ms
root@PC67999:/home/ubuntu#
```

Figure 25: ping 192.168.1.11

The values are :

- Min RTT : 2.835 ms,
- Avg RTT : 3.601 ms
- Max RTT : 6.869 ms
- mDev : 0.968 ms

We have pinged from node 14 to other nodes as well.

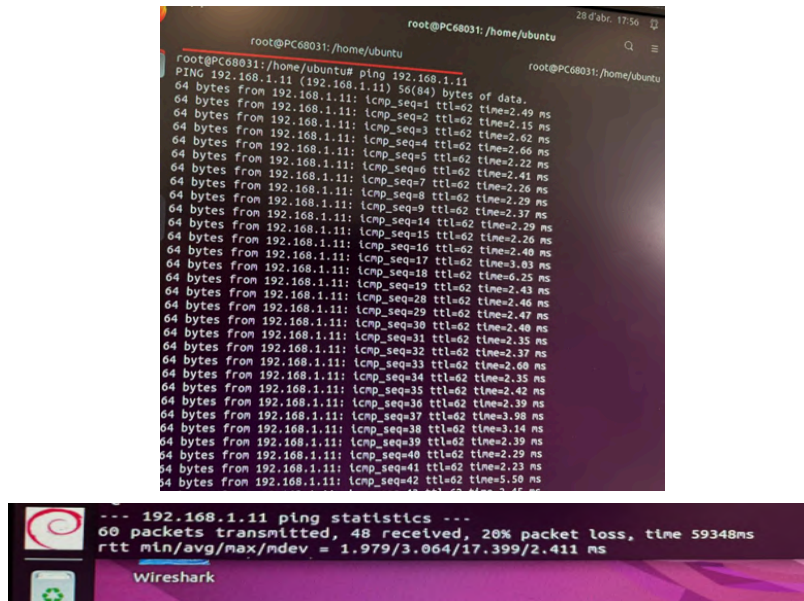


Figure 26: Ping from node 14 to node 11, showing the RTT variation

The values are:

- Min RTT : 1.979 ms,
- Avg RTT : 3.064 ms
- Max RTT : 17.399 ms
- mDev : 2.411 ms

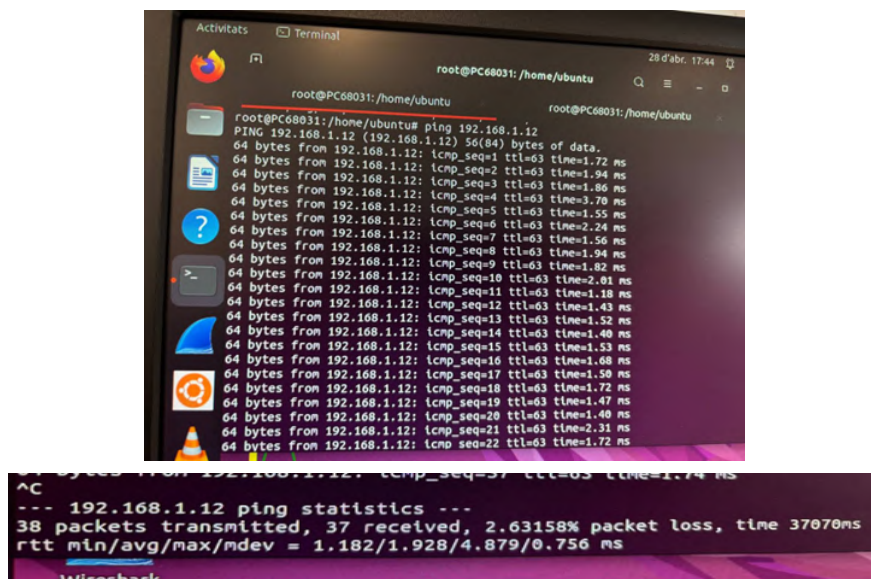


Figure 27: Ping from node 14 to node 12, showing the RTT variation

The values are:

- Min RTT : 1.182 ms,
- Avg RTT : 1.928 ms
- Max RTT : 4.879 ms
- mDev : 0.756 ms

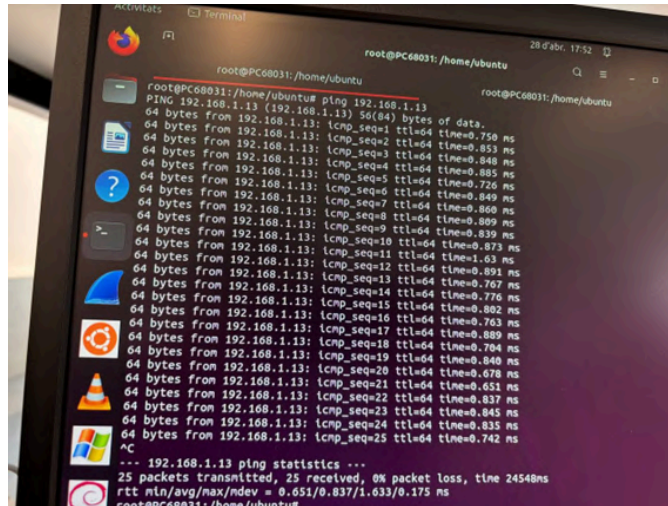


Figure 28: Ping from node 14 to node 12, showing the RTT variation

The values are:

- Min RTT : 0.651 ms,
- Avg RTT : 0.837 ms
- Max RTT : 1.633 ms
- mDev : 0.175 ms

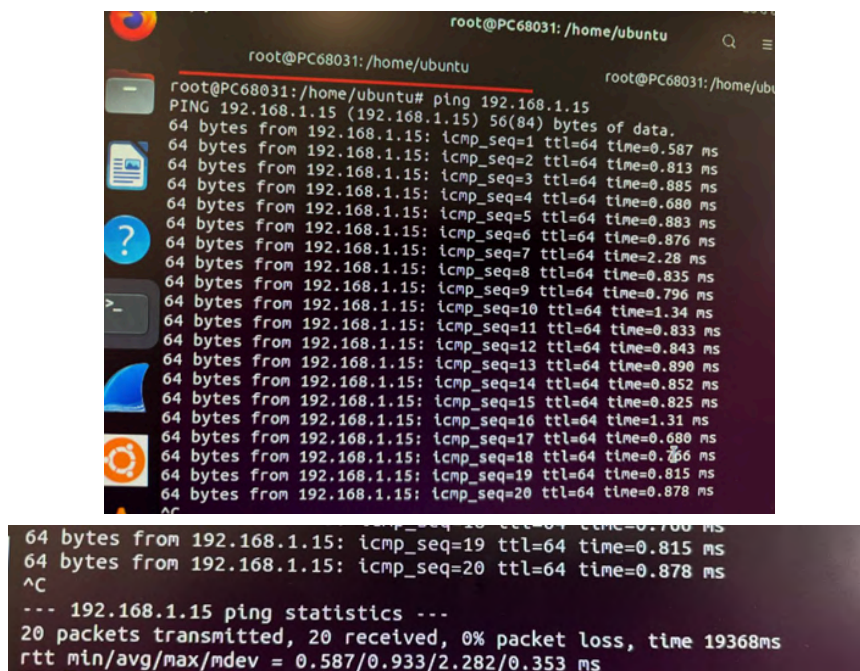


Figure 29: Ping from node 14 to node 15, showing the RTT variation

So the following table shows the summary of arrival times from node 14 :

	Min RTT	Avg RTT	Max RTT	mDev rtt
Node 11	1.979 ms	3.064 ms	17.399 ms	2.411 ms
Node 12	1.182 ms	1.928 ms	4.879 ms	0.756 ms
Node 13	0.651 ms	0.837 ms	1.633 ms	0.175 ms
Node 15	0.587 ms	0.933 ms	2.282 ms	0.353 ms

The following graph represents the RTT measurements obtained from node 14 to the other nodes. Each node is shown with a different color. The results show that direct neighbors, such as nodes 13 and 15, had lower RTT values, while nodes 12 and 11 had higher delay. Node 11 also showed the highest maximum RTT, which indicates more delay variation in the longer multi-hop path.

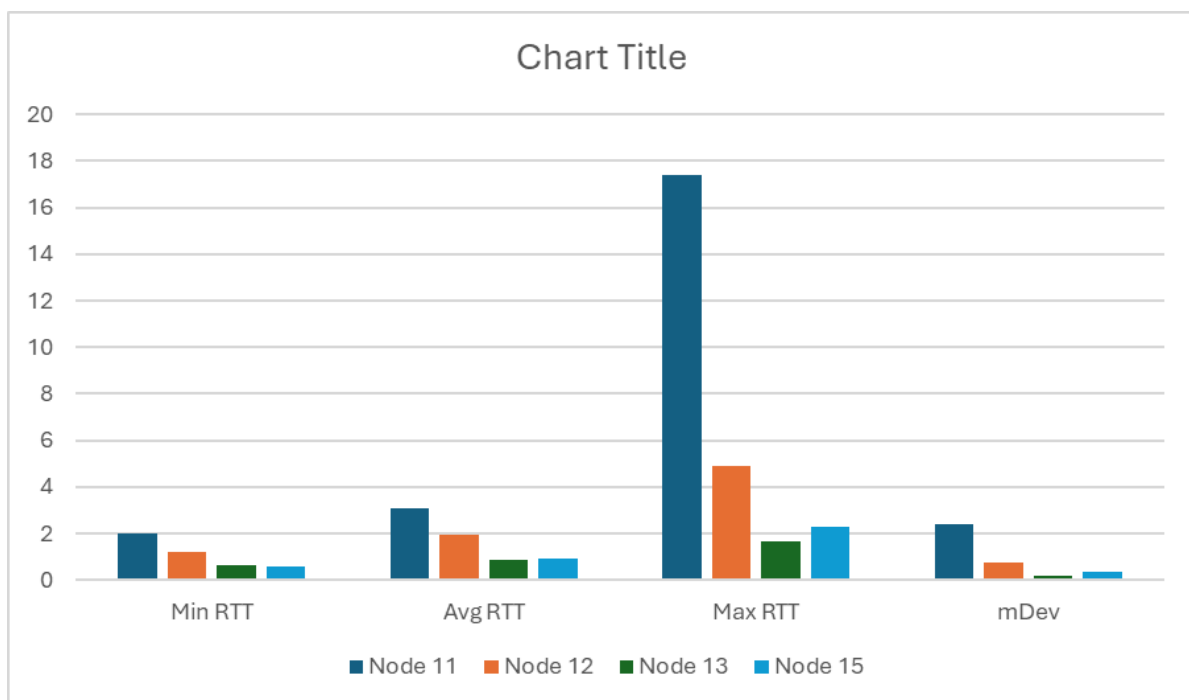


Figure 30 : OLSR RTT measurements from node 14

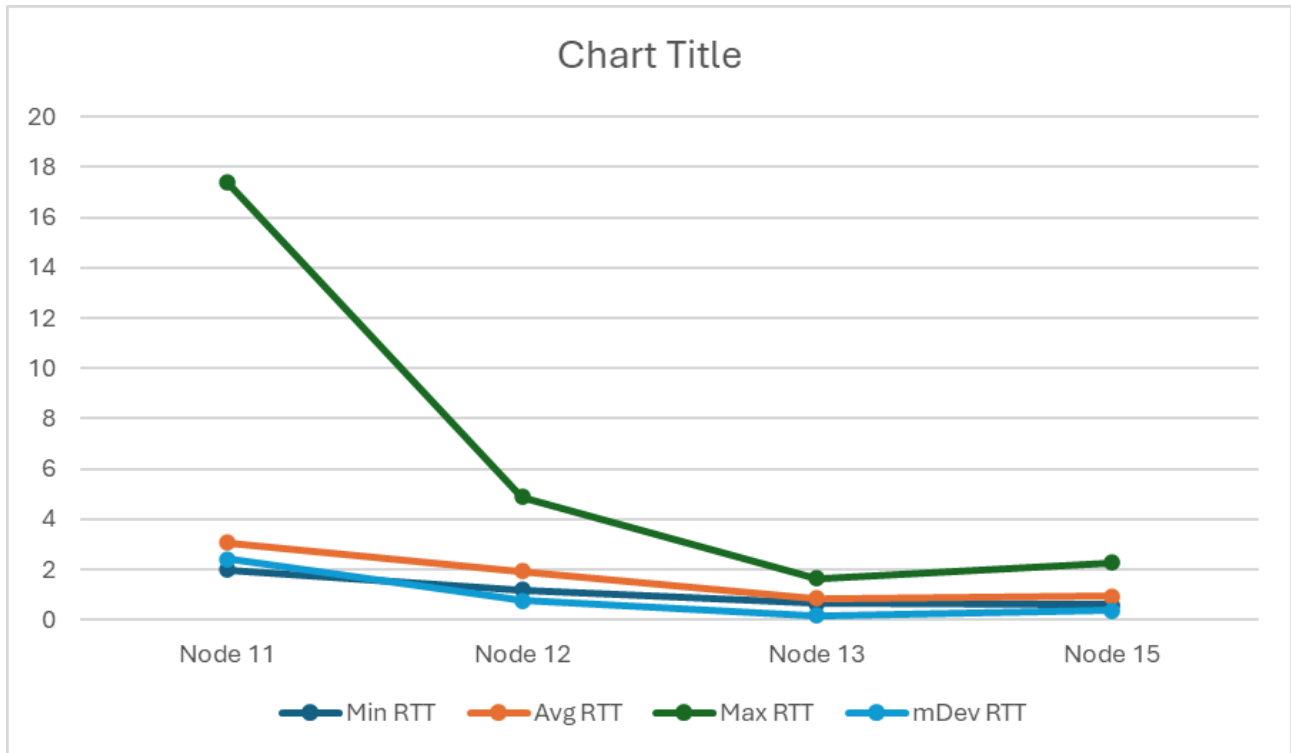


Figure 31 : OLSR RTT metrics from node 14

And the following results show the summary of arrival times from node 15 :

	Min RTT	Avg RTT	Max RTT	mDev rtt
Node 11	2.835 ms	3.601 ms	6.869 ms	0.968 ms
Node 12	1.975 ms	4.133 ms	34.760 ms	6.883 ms
Node 13	1.333 ms	11.073 ms	53.488 ms	14.423 ms
Node 14	0.610 ms	0.825 ms	0.906 ms	0.084 ms

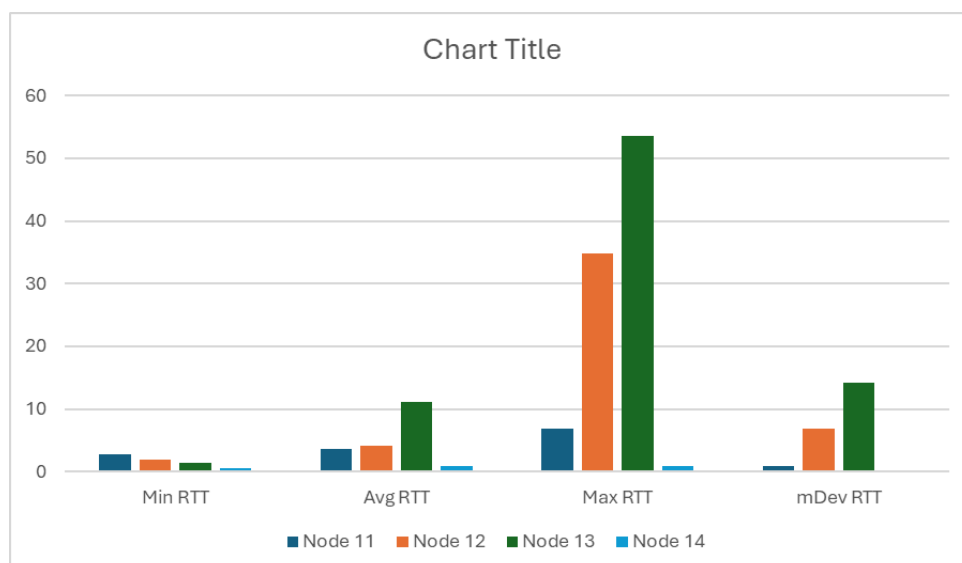


Figure 32 : OLSR RTT measurements from node 15

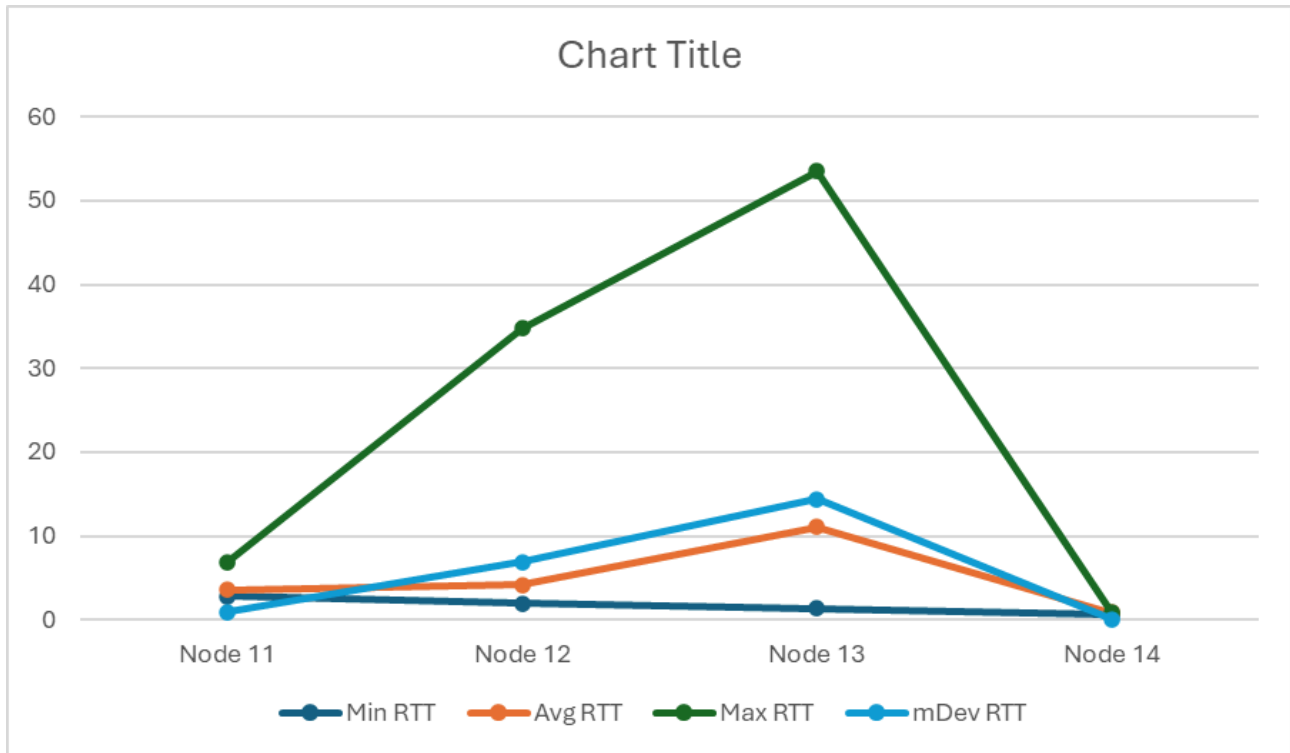


Figure 33 : OLSR RTT metrics from node 15

The results show a clear tendency: the delay increases as the number of hops grows (one hop, two hops, three hops, etc.). The **minimum RTT** follows a fairly linear trend, since it is the least affected by randomness in the transmission process. In contrast, the **average RTT** is less linear because it depends on the variability of all collected samples, which tend to fluctuate over time. The **maximum RTT** shows the most irregular behavior, with noticeable oscillations and peaks, but it still follows an overall increasing trend as the number of hops increases.

This variability is mainly due to the nature of Wi-Fi communications. Wi-Fi does not guarantee a constant delay because it relies on a random backoff mechanism before

transmission. When a node transmits a packet, it may succeed immediately, or it may fail to receive an acknowledgment (ACK), leading to retransmissions. As the number of retries

increases, the delay grows significantly. Therefore, the **RTT can vary considerably**, especially in multi-hop scenarios where each link introduces its own probability of delay and retransmission.

The minimum RTT is less affected by these random effects, which explains its linear behavior. On the other hand, the maximum RTT reflects worst-case situations, often caused by bad luck in channel access or multiple retransmissions, resulting in strong oscillations. These peaks also influence the average RTT, making it less stable.

Another important factor is the presence of additional network traffic. Nodes are not only sending ping packets but also control messages from routing protocols such as OLSR. By default, OLSR generates Hello messages approximately every two seconds. This means that both ping traffic and routing protocol messages share the same wireless medium.

Although collisions between packets (e.g., ping and Hello messages) are possible, their probability is generally low under normal conditions. This is because packet transmission times are extremely short (on the order of microseconds), and each node uses the channel only for a very small fraction of time. Therefore, even if a node sends one ping per second and another sends Hello messages every two seconds, the likelihood of collision remains limited.

However, if the frequency of control messages increases (for example, if OLSR messages are sent more often), the probability of collisions rises. This leads to more retransmissions and, consequently, higher RTT values—especially noticeable in the maximum RTT measurements. Despite these peaks and fluctuations, the network is still able to function correctly, although with increased delay variability.

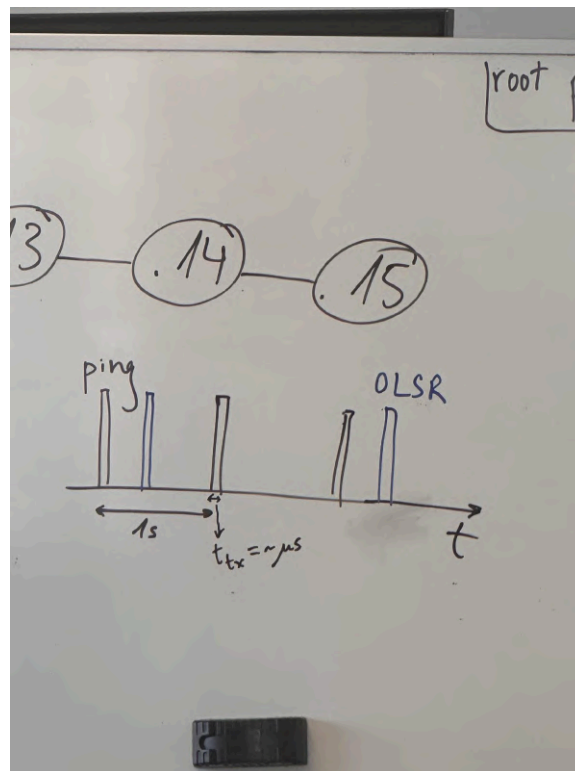


Figure 34: Channel occupancy of Ping and OLSR packets

In the second part, we transform the first scenario into a new one (In this setup, the routing protocol has two possible paths to choose from, each with an associated cost. Initially, the lower path is preferred because it has a better (lower) cost than the upper one. However, when node fifteen is removed, the OLSR detects the failure and

automatically recomputes the routes, selecting an alternative path to maintain network connectivity.

Change the iptables -t mangle -D

```

192.168.1.11: icmp_seq=11 ttl=61 time=3.03 ms
192.168.1.11: icmp_seq=12 ttl=61 time=3.57 ms
192.168.1.11: icmp_seq=13 ttl=61 time=3.00 ms

root@PC67999: /home/ubuntu
*** olsr.org - 0.6.6.2-git_0000000-hash_12fa41b5362519d37bea6715ce291978
(2014-09-11 13:09:19 on toyol) ***

--- 18:01:29.987007 ----- LINKS

IP address      hyst      LQ      ETX
192.168.1.14    0.000    1.000/1.000  1.000
192.168.1.11    0.000    0.886/0.686  1.644

--- 18:01:29.987021 ----- NEIGHBORS

IP address      LQ      NLQ      SYM      MPR      MPRS      will
192.168.1.11    0.000    YES      YES      YES      3
192.168.1.14    0.000    YES      YES      YES      3

--- 18:01:29.987028 ----- TWO-HOP NEIGHBORS

IP addr (2-hop) IP addr (1-hop) Total cost
192.168.1.12    192.168.1.11    2.644
192.168.1.13    192.168.1.14    2.000

```

Figure 35: OLSR routing table

We start by initiating a ping between nodes 14 and 11. Then, we disconnect the Wi-Fi device and leave it unplugged for a short period of time.

Expected result that should appear.

12 13

11-15-14

The expected result is that node fifteen remains considered “alive” for a short period after it actually disappears. This happens because neighboring nodes continue sending Hello messages every two seconds, according to the OLSR configuration. When node fifteen is removed, it does not disappear immediately from the network’s perspective. Instead, a node detects the link failure only after three consecutive Hello messages are missed. During this time, packets may still be sent to a node that is no longer present, resulting in packet loss and a temporary connectivity gap. The failure detection time depends on when the node disappears relative to the Hello interval, so it falls within a range of approximately two to three Hello intervals. Since the default Hello interval is about 2 seconds, the total recovery time is typically between 4 and 6 seconds. This delay

represents the time required for the protocol to detect the failure and recompute an alternative route.

(NOT GOTTEN THE RESULT. NOT A PICTURE)

The measured recovery time (RCL) in the previous experiment was 7.2 seconds. This raises the question of whether we can reduce this time by adjusting parameters. In OLSR, reducing the Hello interval can indeed shorten the failure detection and recovery time. However, it is not feasible to decrease the Hello interval indefinitely (e.g., close to zero), because this would generate an excessive number of control messages and consume a large portion of the network capacity. Therefore, performance can be improved by reducing the Hello interval, but only up to a certain practical limit. Additionally, since OLSR operates as an application-level protocol, there may be extra delays due to processing and system response times, which explains why the measured recovery time can differ slightly from the theoretical values.

Session 3. AODV

Each group must decide which routing protocol to use, such as AODV or the alternative option, based on the experimental results obtained during the practical part. Since this is an open question, there is no single correct answer; the choice should be justified according to the performance observed in the experiments.

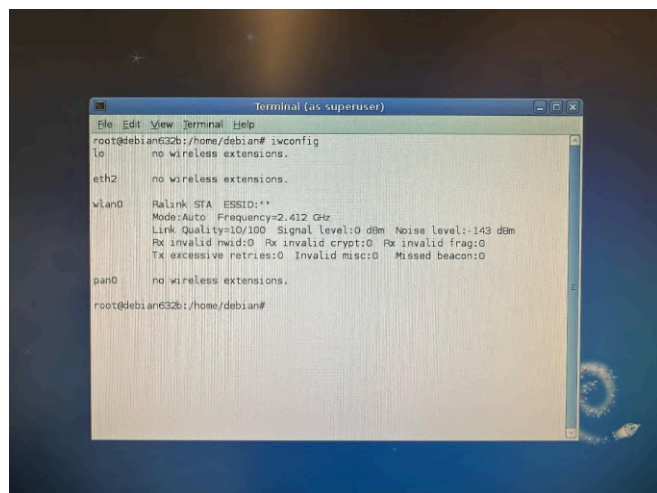
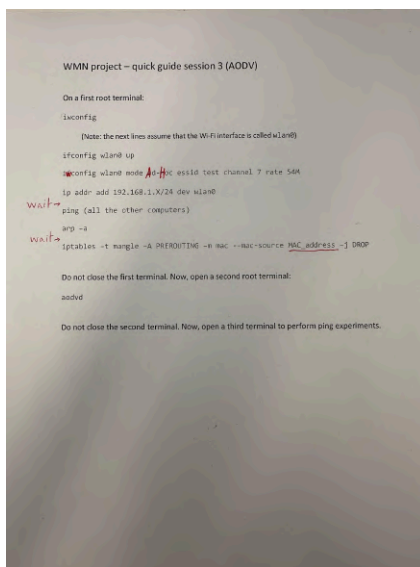
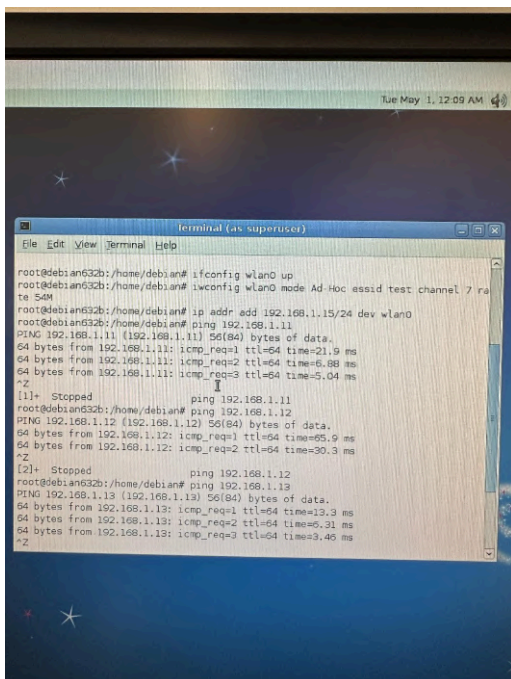


Figure 36 & 37 : AODV setup commands used to configure the Wi-Fi interface in Ad-Hoc mode

First, the Wi-Fi card must be configured in Ad-Hoc mode instead of Infrastructure mode, which is normally used to join an access point. The wireless interface wlan0 is activated with the command **ifconfig wlan0 up**, causing the Wi-Fi card to start blinking, indicating that it is enabled. Then, the node joins the Ad-Hoc network on channel 7 with a bit rate of

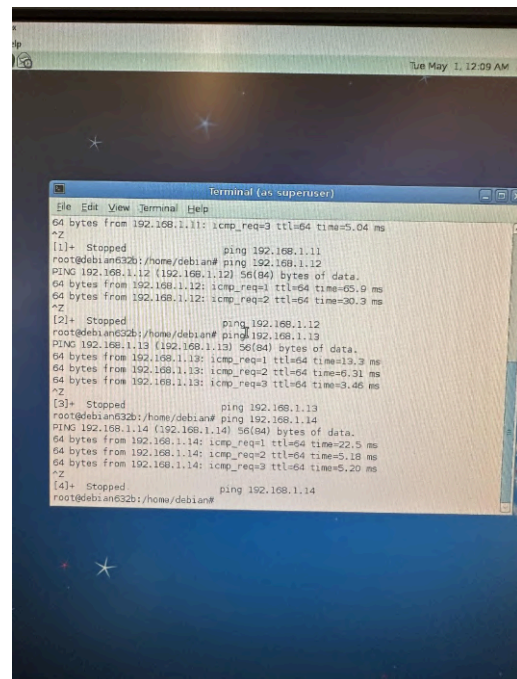
54 Mbps using the command **iwconfig wlan0 mode Ad-Hoc essid test channel 7 rate 54M**. After configuring the wireless parameters, the IP address is assigned in the same way as in the previous session. In our case, the address 192.168.1.15/24 is configured with the command **ip addr add 192.168.1.15/24 dev wlan0**. Once the network interface is configured, connectivity with the other nodes in the same network is verified using ping commands. The devices that respond directly are considered one-hop neighbors. The **arp -a** command can also be used to identify the MAC addresses of neighboring nodes. Finally, iptables rules may be applied to block specific links and create different topologies for the experiments, while the aodvd daemon is executed in a second terminal and a third terminal is used to perform ping experiments and analyze the routing behavior of the AODV protocol.



```

root@debian632b:/home/debian# ifconfig wlan0 up
root@debian632b:/home/debian# iwconfig wlan0 mode Ad-Hoc essid test channel 7 rate 54M
root@debian632b:/home/debian# ip addr add 192.168.1.15/24 dev wlan0
root@debian632b:/home/debian# ping 192.168.1.11
PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data:
64 bytes from 192.168.1.11: icmp_req=1 ttl=64 time=21.9 ms
64 bytes from 192.168.1.11: icmp_req=2 ttl=64 time=6.88 ms
64 bytes from 192.168.1.11: icmp_req=3 ttl=64 time=5.04 ms
^C
[1]+  Stopped                  ping 192.168.1.11
root@debian632b:/home/debian# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data:
64 bytes from 192.168.1.12: icmp_req=1 ttl=64 time=65.9 ms
64 bytes from 192.168.1.12: icmp_req=2 ttl=64 time=30.3 ms
^C
[2]+  Stopped                  ping 192.168.1.12
root@debian632b:/home/debian# ping 192.168.1.13
PING 192.168.1.13 (192.168.1.13) 56(84) bytes of data:
64 bytes from 192.168.1.13: icmp_req=1 ttl=64 time=13.3 ms
64 bytes from 192.168.1.13: icmp_req=2 ttl=64 time=5.31 ms
64 bytes from 192.168.1.13: icmp_req=3 ttl=64 time=3.45 ms
^C
[3]+  Stopped                  ping 192.168.1.13
root@debian632b:/home/debian# ping 192.168.1.14
PING 192.168.1.14 (192.168.1.14) 56(84) bytes of data:
64 bytes from 192.168.1.14: icmp_req=1 ttl=64 time=22.5 ms
64 bytes from 192.168.1.14: icmp_req=2 ttl=64 time=5.18 ms
64 bytes from 192.168.1.14: icmp_req=3 ttl=64 time=5.20 ms
^C
[4]+  Stopped                  ping 192.168.1.14
root@debian632b:/home/debian#

```



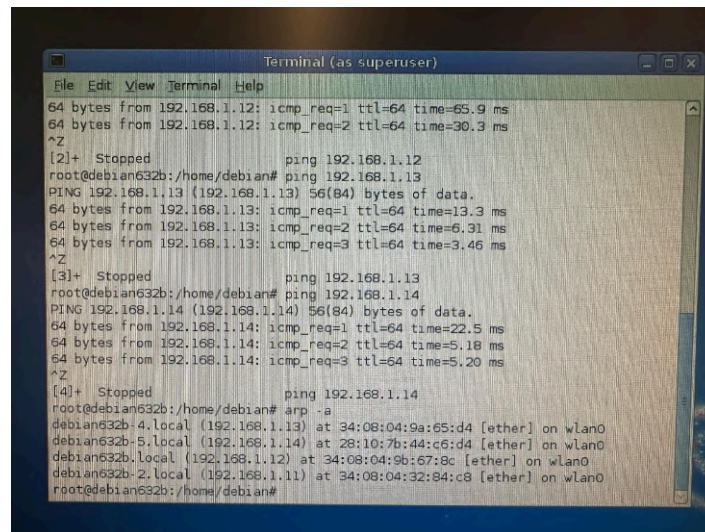
```

root@debian632b:/home/debian# ping 192.168.1.11
PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data:
64 bytes from 192.168.1.11: icmp_req=1 ttl=64 time=5.04 ms
^C
[1]+  Stopped                  ping 192.168.1.11
root@debian632b:/home/debian# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data:
64 bytes from 192.168.1.12: icmp_req=1 ttl=64 time=65.9 ms
64 bytes from 192.168.1.12: icmp_req=2 ttl=64 time=30.3 ms
^C
[2]+  Stopped                  ping 192.168.1.12
root@debian632b:/home/debian# ping 192.168.1.13
PING 192.168.1.13 (192.168.1.13) 56(84) bytes of data:
64 bytes from 192.168.1.13: icmp_req=1 ttl=64 time=13.3 ms
64 bytes from 192.168.1.13: icmp_req=2 ttl=64 time=5.31 ms
64 bytes from 192.168.1.13: icmp_req=3 ttl=64 time=3.45 ms
^C
[3]+  Stopped                  ping 192.168.1.13
root@debian632b:/home/debian# ping 192.168.1.14
PING 192.168.1.14 (192.168.1.14) 56(84) bytes of data:
64 bytes from 192.168.1.14: icmp_req=1 ttl=64 time=22.5 ms
64 bytes from 192.168.1.14: icmp_req=2 ttl=64 time=5.18 ms
64 bytes from 192.168.1.14: icmp_req=3 ttl=64 time=5.20 ms
^C
[4]+  Stopped                  ping 192.168.1.14
root@debian632b:/home/debian#

```

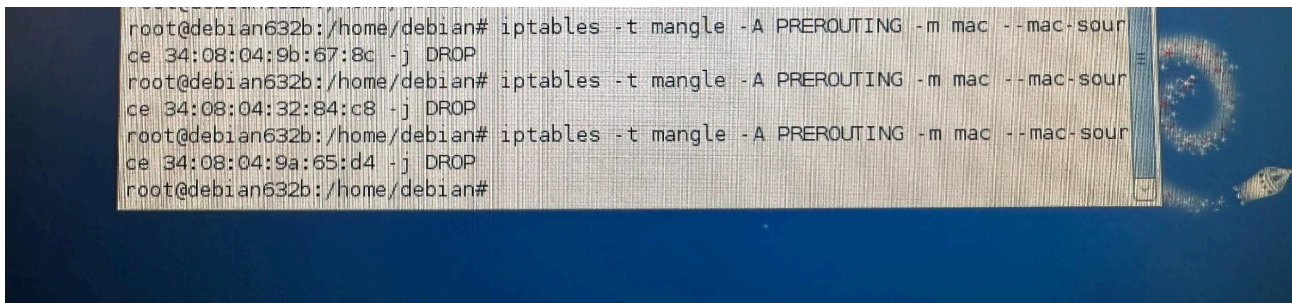
Figures 38 and 39 : Connectivity test between node 15 and other nodes

All the nodes are one-hop neighbors, meaning that they can communicate directly with each other and have connectivity with the rest of the devices in the network. The command **arp -a** is used to display the ARP cache, which contains the correspondence between IP addresses and MAC layer addresses. In other words, the ARP protocol is responsible for determining which MAC address corresponds to a given IP address. As a result, the command provides a list of the IP and MAC addresses of all the devices currently detected in the local network.



```

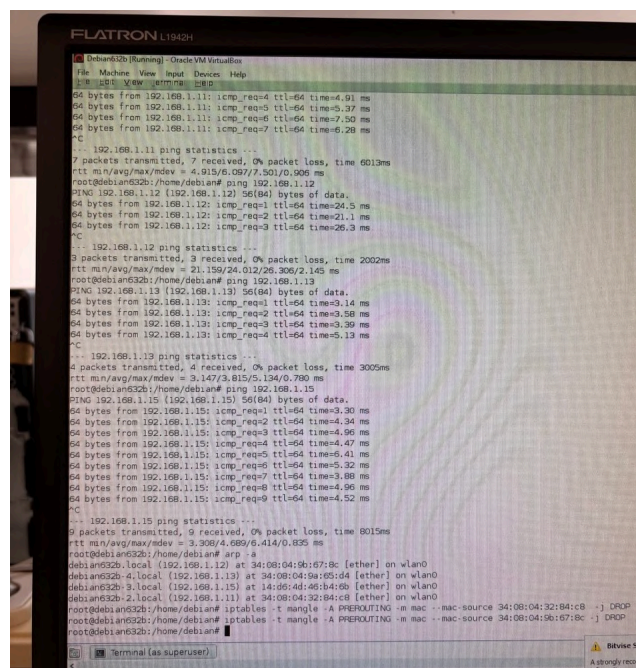
Terminal (as superuser)
64 bytes from 192.168.1.12: icmp_req=1 ttl=64 time=65.9 ms
64 bytes from 192.168.1.12: icmp_req=2 ttl=64 time=30.3 ms
^Z
[2]+  Stopped                  ping 192.168.1.12
root@debian632b:/home/debian# ping 192.168.1.13
PING 192.168.1.13 (192.168.1.13) 56(84) bytes of data.
64 bytes from 192.168.1.13: icmp_req=1 ttl=64 time=13.3 ms
64 bytes from 192.168.1.13: icmp_req=2 ttl=64 time=6.31 ms
64 bytes from 192.168.1.13: icmp_req=3 ttl=64 time=3.46 ms
^Z
[3]+  Stopped                  ping 192.168.1.13
root@debian632b:/home/debian# ping 192.168.1.14
PING 192.168.1.14 (192.168.1.14) 56(84) bytes of data.
64 bytes from 192.168.1.14: icmp_req=1 ttl=64 time=22.5 ms
64 bytes from 192.168.1.14: icmp_req=2 ttl=64 time=5.18 ms
64 bytes from 192.168.1.14: icmp_req=3 ttl=64 time=5.20 ms
^Z
[4]+  Stopped                  ping 192.168.1.14
root@debian632b:/home/debian# arp -a
debian632b-4.local (192.168.1.13) at 34:08:04:9a:65:d4 [ether] on wlan0
debian632b-5.local (192.168.1.14) at 28:10:7b:44:c6:d4 [ether] on wlan0
debian632b.local (192.168.1.12) at 34:08:04:9b:67:8c [ether] on wlan0
debian632b-2.local (192.168.1.11) at 34:08:04:32:84:c8 [ether] on wlan0
root@debian632b:/home/debian#
  
```



```

root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:9b:67:8c -j DROP
root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:32:84:c8 -j DROP
root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:9a:65:d4 -j DROP
root@debian632b:/home/debian#
  
```

Figures 40 and 41 : Initial AODV neighbor verification and MAC address information.



```

FLATRON L1942H
Debian632b (Running) - Oracle VM VirtualBox
File Machine View Input Devices Help
File Edit View Terminal Help
64 bytes from 192.168.1.11: icmp_req=4 ttl=64 time=4.91 ms
64 bytes from 192.168.1.11: icmp_req=5 ttl=64 time=5.37 ms
64 bytes from 192.168.1.11: icmp_req=6 ttl=64 time=7.50 ms
64 bytes from 192.168.1.11: icmp_req=7 ttl=64 time=6.28 ms
^C
-- 192.168.1.11 ping statistics --
7 packets transmitted, 7 received, 0% packet loss, time 6013ms
rtt min/avg/max/ndev = 4.915/6.097/7.501/0.906 ms
root@debian632b:/home/debian# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data.
64 bytes from 192.168.1.12: icmp_req=1 ttl=64 time=24.5 ms
64 bytes from 192.168.1.12: icmp_req=2 ttl=64 time=21.1 ms
64 bytes from 192.168.1.12: icmp_req=3 ttl=64 time=26.3 ms
^C
-- 192.168.1.12 ping statistics --
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/ndev = 21.159/24.012/26.306/2.145 ms
root@debian632b:/home/debian# ping 192.168.1.13
PING 192.168.1.13 (192.168.1.13) 56(84) bytes of data.
64 bytes from 192.168.1.13: icmp_req=1 ttl=64 time=9.14 ms
64 bytes from 192.168.1.13: icmp_req=2 ttl=64 time=3.58 ms
64 bytes from 192.168.1.13: icmp_req=3 ttl=64 time=3.39 ms
64 bytes from 192.168.1.13: icmp_req=4 ttl=64 time=5.13 ms
^C
-- 192.168.1.13 ping statistics --
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/ndev = 3.147/3.815/5.194/0.780 ms
root@debian632b:/home/debian# ping 192.168.1.15
PING 192.168.1.15 (192.168.1.15) 56(84) bytes of data.
64 bytes from 192.168.1.15: icmp_req=1 ttl=64 time=3.30 ms
64 bytes from 192.168.1.15: icmp_req=2 ttl=64 time=4.34 ms
64 bytes from 192.168.1.15: icmp_req=3 ttl=64 time=4.96 ms
64 bytes from 192.168.1.15: icmp_req=4 ttl=64 time=4.47 ms
64 bytes from 192.168.1.15: icmp_req=5 ttl=64 time=5.41 ms
64 bytes from 192.168.1.15: icmp_req=6 ttl=64 time=5.32 ms
64 bytes from 192.168.1.15: icmp_req=7 ttl=64 time=3.88 ms
64 bytes from 192.168.1.15: icmp_req=8 ttl=64 time=4.86 ms
64 bytes from 192.168.1.15: icmp_req=9 ttl=64 time=4.52 ms
^C
-- 192.168.1.15 ping statistics --
9 packets transmitted, 9 received, 0% packet loss, time 8015ms
rtt min/avg/max/ndev = 3.308/4.689/6.414/0.835 ms
root@debian632b:/home/debian# arp -a
debian632b.local (192.168.1.12) at 34:08:04:9b:67:8c [ether] on wlan0
debian632b-4.local (192.168.1.13) at 34:08:04:9a:65:d4 [ether] on wlan0
debian632b-3.local (192.168.1.15) at 14:06:ad:46:b4:b6 [ether] on wlan0
debian632b-2.local (192.168.1.11) at 34:08:04:32:84:c8 [ether] on wlan0
root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:32:84:c8 -j DROP
root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:9b:67:8c -j DROP
root@debian632b:/home/debian#
  
```

Figure 42 : Node 14 pinging other nodes

```

root@debian632b:/home/debian# ifconfig wlan0 192.168.1.12 netmask 255.255.255.0
root@debian632b:/home/debian# ifconfig wlan0 192.168.1.13 netmask 255.255.255.0
root@debian632b:/home/debian# ifconfig wlan0 192.168.1.15 netmask 255.255.255.0
root@debian632b:/home/debian# ifconfig wlan0 192.168.1.11 netmask 255.255.255.0
root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:32:84:c8 -j DROP
root@debian632b:/home/debian# iptables -t mangle -A PREROUTING -m mac --mac-source 34:08:04:9b:67:8c -j DROP
root@debian632b:/home/debian#

```

Figure 43 : MAC filtering rules used to force the multi-hop topology. (node 14 POV)

We want to emulate a specific network scenario by controlling which nodes are allowed to communicate directly. To achieve this, the iptables command introduced in the previous session is used to discard traffic coming from nodes that are not considered direct neighbors. In this case, traffic from nodes 11, 12, 13, and 14 is blocked in order to modify the network topology and force the routing protocol to discover alternative paths. This is done by applying filtering rules based on the MAC addresses of the neighboring devices, allowing only communication with the intended direct neighbors while discarding packets from the others.

```

Tue May 1, 12:21 AM
Terminal (as superuser)
root@debian632b:/home/debian# aodvvd
bash: aodvvd: command not found
root@debian632b:/home/debian# aodvd
00:21:11.025 host_init: Attaching to wlan0, override with -i <if1,if2,...>.
00:21:11.130 aodv_socket_init: RAW send socket buffer size set to 262142
00:21:11.130 aodv_socket_init: Receive buffer size set to 262142
00:21:11.130 main: In wait on reboot for 15000 milliseconds. Disable with "-D".
00:21:11.130 hello_start: Starting to send HELLOs!
00:21:26.131 wait_on_reboot_timeout: Wait on reboot over!!
00:21:48.362 rt_table_insert: Inserting 192.168.1.14 (bucket 0) next hop 192.168.1.14
00:21:48.362 nl_send_add_route_msg: ADD/UPDATE: 192.168.1.14:192.168.1.14 ifindex=3
-mac- 00:21:48.362 rt_table_insert: New timer for 192.168.1.14, life=2100
-mac- 00:21:48.362 hello_process: 192.168.1.14 new NEIGHBOR!
-mac-
-mac-
-mac-

```

Figure 44 : AODV route discovery after applying the filtering rules.

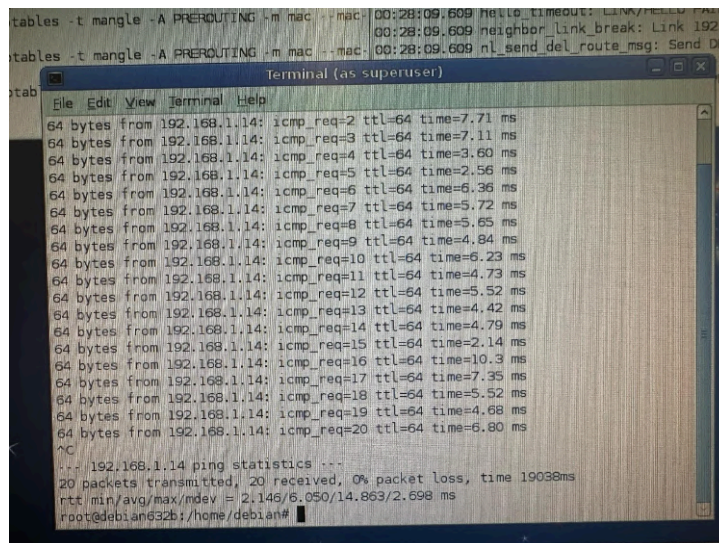
Neighbor .14

In order to send packets to nodes that are not direct neighbors, the devices must first discover a valid route through the network. This is precisely the purpose of the AODV routing protocol, which dynamically discovers and maintains routes between nodes in an Ad-Hoc network. After applying the iptables rules and creating the desired topology, AODV should still allow communication with all the nodes in the network by finding multi-hop routes through intermediate devices. Therefore, even if two nodes cannot communicate directly, packets can still be delivered by forwarding them across neighboring nodes until they reach the final destination.

End to end latency experiment.

In the end-to-end latency experiment, the objective is to measure the time required for a packet to travel from the source node to the destination node and back. This value is obtained using the ping command, which provides the Round-Trip Time, or RTT, associated with each transmission. The measurements are repeated for paths of different lengths: 1 hop, 2 hops, 3 hops, and 4 hops. For each number of hops, 20 RTT samples are collected using the ping command, and the experiment is stopped with Ctrl + C. At the end, ping displays a summary of the statistics, including the minimum, maximum, and average RTT values. These results are then compared and discussed in order to analyze how the end-to-end latency changes as the number of hops increases. In general, a higher number of hops is expected to produce a higher average RTT, because packets must be forwarded through more intermediate nodes before reaching the destination.

Ping 192.168.1.14.



```

tables -t mangle -A PREROUTING -m mac --mac-addr 00:28:09:609: Hello timeout: Link 192.1
tables -t mangle -A PREROUTING -m mac --mac-addr 00:28:09:609: neighbor_link_break: Link 192.1
tables -t mangle -A PREROUTING -m mac --mac-addr 00:28:09:609: nl_send_del_route_msg: Send DE
tab

File Edit View Terminal Help
64 bytes from 192.168.1.14: icmp_req=2 ttl=64 time=7.71 ms
64 bytes from 192.168.1.14: icmp_req=3 ttl=64 time=7.11 ms
64 bytes from 192.168.1.14: icmp_req=4 ttl=64 time=3.60 ms
64 bytes from 192.168.1.14: icmp_req=5 ttl=64 time=2.56 ms
64 bytes from 192.168.1.14: icmp_req=6 ttl=64 time=6.36 ms
64 bytes from 192.168.1.14: icmp_req=7 ttl=64 time=5.72 ms
64 bytes from 192.168.1.14: icmp_req=8 ttl=64 time=5.65 ms
64 bytes from 192.168.1.14: icmp_req=9 ttl=64 time=4.84 ms
64 bytes from 192.168.1.14: icmp_req=10 ttl=64 time=6.23 ms
64 bytes from 192.168.1.14: icmp_req=11 ttl=64 time=4.73 ms
64 bytes from 192.168.1.14: icmp_req=12 ttl=64 time=5.52 ms
64 bytes from 192.168.1.14: icmp_req=13 ttl=64 time=4.42 ms
64 bytes from 192.168.1.14: icmp_req=14 ttl=64 time=4.79 ms
64 bytes from 192.168.1.14: icmp_req=15 ttl=64 time=2.14 ms
64 bytes from 192.168.1.14: icmp_req=16 ttl=64 time=10.3 ms
64 bytes from 192.168.1.14: icmp_req=17 ttl=64 time=7.35 ms
64 bytes from 192.168.1.14: icmp_req=18 ttl=64 time=5.52 ms
64 bytes from 192.168.1.14: icmp_req=19 ttl=64 time=4.68 ms
64 bytes from 192.168.1.14: icmp_req=20 ttl=64 time=6.80 ms
^C
... 192.168.1.14 ping statistics ...
20 packets transmitted, 20 received, 0% packet loss, time 19038ms
rtt min/avg/max/mdev = 2.146/6.050/14.863/2.698 ms
root@debian632b:/home/debian#

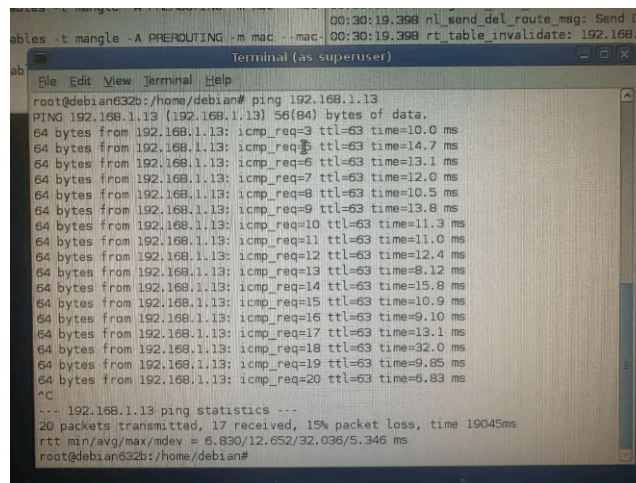
```

Figure 45 : AODV ping from node 15 to node 14 with RTT variation

The values are:

- Min RTT : 2.146 ms,
- Avg RTT : 6.050 ms
- Max RTT : 14.863 ms
- mDev : 2.698 ms

Ping 192.168.1.13



```

root@debian632b:/home/debian# ping 192.168.1.13
PING 192.168.1.13 (192.168.1.13) 56(84) bytes of data:
64 bytes from 192.168.1.13: icmp_req=3 ttl=63 time=10.0 ms
64 bytes from 192.168.1.13: icmp_req=4 ttl=63 time=14.7 ms
64 bytes from 192.168.1.13: icmp_req=5 ttl=63 time=13.1 ms
64 bytes from 192.168.1.13: icmp_req=6 ttl=63 time=12.0 ms
64 bytes from 192.168.1.13: icmp_req=7 ttl=63 time=10.5 ms
64 bytes from 192.168.1.13: icmp_req=8 ttl=63 time=13.8 ms
64 bytes from 192.168.1.13: icmp_req=9 ttl=63 time=11.3 ms
64 bytes from 192.168.1.13: icmp_req=10 ttl=63 time=11.0 ms
64 bytes from 192.168.1.13: icmp_req=11 ttl=63 time=12.4 ms
64 bytes from 192.168.1.13: icmp_req=12 ttl=63 time=8.12 ms
64 bytes from 192.168.1.13: icmp_req=13 ttl=63 time=15.8 ms
64 bytes from 192.168.1.13: icmp_req=14 ttl=63 time=10.9 ms
64 bytes from 192.168.1.13: icmp_req=15 ttl=63 time=9.10 ms
64 bytes from 192.168.1.13: icmp_req=16 ttl=63 time=13.1 ms
64 bytes from 192.168.1.13: icmp_req=17 ttl=63 time=32.0 ms
64 bytes from 192.168.1.13: icmp_req=18 ttl=63 time=9.85 ms
64 bytes from 192.168.1.13: icmp_req=19 ttl=63 time=6.83 ms
64 bytes from 192.168.1.13: icmp_req=20 ttl=63 time=6.83 ms
^C
--- 192.168.1.13 ping statistics ---
20 packets transmitted, 17 received, 15% packet loss, time 19045ms
rtt min/avg/max/mdev = 6.830/12.652/32.036/5.346 ms
root@debian632b:/home/debian#

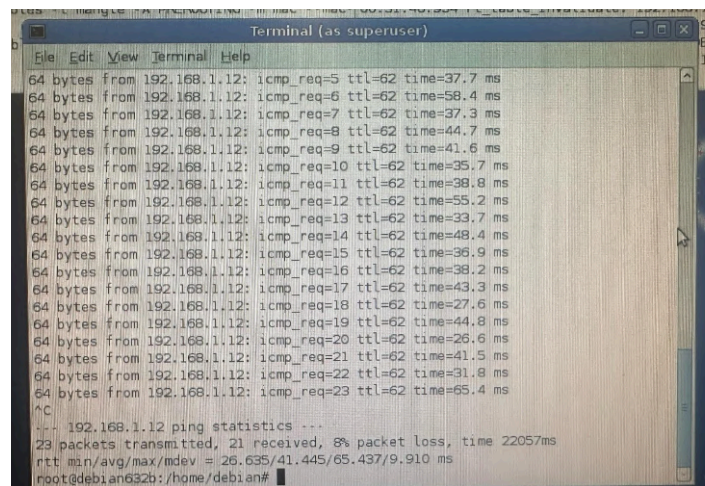
```

Figure 46 : AODV ping from node 15 to node 13 with RTT variation

The values are:

- Min RTT : 6.830 ms,
- Avg RTT : 12.652 ms
- Max RTT : 32.036 ms
- mDev : 5.346 ms

Ping 192.168.1.12



```

root@debian632b:/home/debian# ping 192.168.1.12
64 bytes from 192.168.1.12: icmp_req=5 ttl=62 time=37.7 ms
64 bytes from 192.168.1.12: icmp_req=6 ttl=62 time=58.4 ms
64 bytes from 192.168.1.12: icmp_req=7 ttl=62 time=37.3 ms
64 bytes from 192.168.1.12: icmp_req=8 ttl=62 time=44.7 ms
64 bytes from 192.168.1.12: icmp_req=9 ttl=62 time=41.6 ms
64 bytes from 192.168.1.12: icmp_req=10 ttl=62 time=35.7 ms
64 bytes from 192.168.1.12: icmp_req=11 ttl=62 time=38.8 ms
64 bytes from 192.168.1.12: icmp_req=12 ttl=62 time=55.2 ms
64 bytes from 192.168.1.12: icmp_req=13 ttl=62 time=33.7 ms
64 bytes from 192.168.1.12: icmp_req=14 ttl=62 time=48.4 ms
64 bytes from 192.168.1.12: icmp_req=15 ttl=62 time=36.9 ms
64 bytes from 192.168.1.12: icmp_req=16 ttl=62 time=38.2 ms
64 bytes from 192.168.1.12: icmp_req=17 ttl=62 time=43.3 ms
64 bytes from 192.168.1.12: icmp_req=18 ttl=62 time=27.6 ms
64 bytes from 192.168.1.12: icmp_req=19 ttl=62 time=44.8 ms
64 bytes from 192.168.1.12: icmp_req=20 ttl=62 time=26.6 ms
64 bytes from 192.168.1.12: icmp_req=21 ttl=62 time=41.5 ms
64 bytes from 192.168.1.12: icmp_req=22 ttl=62 time=31.8 ms
64 bytes from 192.168.1.12: icmp_req=23 ttl=62 time=65.4 ms
^C
--- 192.168.1.12 ping statistics ---
23 packets transmitted, 21 received, 8% packet loss, time 22057ms
rtt min/avg/max/mdev = 26.635/41.445/65.437/9.910 ms
root@debian632b:/home/debian#

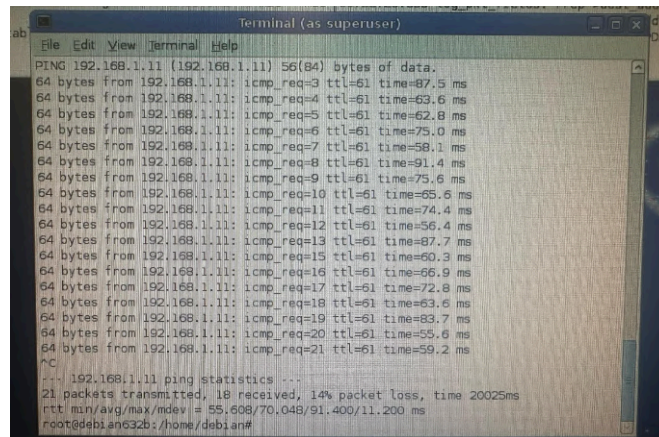
```

Figure 47 : AODV ping from node 15 to node 12 with RTT variation

The values are:

- Min RTT : 26.635 ms,
- Avg RTT : 41.445 ms
- Max RTT : 65.437 ms
- mDev : 9.910 ms

Ping 192.168.1.11



```

Terminal (as superuser)
PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
64 bytes from 192.168.1.11: icmp_req=3 ttl=61 time=87.5 ms
64 bytes from 192.168.1.11: icmp_req=4 ttl=61 time=63.6 ms
64 bytes from 192.168.1.11: icmp_req=5 ttl=61 time=62.8 ms
64 bytes from 192.168.1.11: icmp_req=6 ttl=61 time=75.0 ms
64 bytes from 192.168.1.11: icmp_req=7 ttl=61 time=58.1 ms
64 bytes from 192.168.1.11: icmp_req=8 ttl=61 time=91.4 ms
64 bytes from 192.168.1.11: icmp_req=9 ttl=61 time=75.6 ms
64 bytes from 192.168.1.11: icmp_req=10 ttl=61 time=65.6 ms
64 bytes from 192.168.1.11: icmp_req=11 ttl=61 time=74.4 ms
64 bytes from 192.168.1.11: icmp_req=12 ttl=61 time=56.4 ms
64 bytes from 192.168.1.11: icmp_req=13 ttl=61 time=87.7 ms
64 bytes from 192.168.1.11: icmp_req=15 ttl=61 time=60.3 ms
64 bytes from 192.168.1.11: icmp_req=16 ttl=61 time=66.9 ms
64 bytes from 192.168.1.11: icmp_req=17 ttl=61 time=72.8 ms
64 bytes from 192.168.1.11: icmp_req=18 ttl=61 time=63.6 ms
64 bytes from 192.168.1.11: icmp_req=19 ttl=61 time=83.7 ms
64 bytes from 192.168.1.11: icmp_req=20 ttl=61 time=55.6 ms
64 bytes from 192.168.1.11: icmp_req=21 ttl=61 time=59.2 ms
^C
--- 192.168.1.11 ping statistics ---
21 packets transmitted, 18 received, 14% packet loss, time 20025ms
rtt min/avg/max/mdev = 55.608/70.048/91.400/11.200 ms
root@debian632b: /home/debian#
  
```

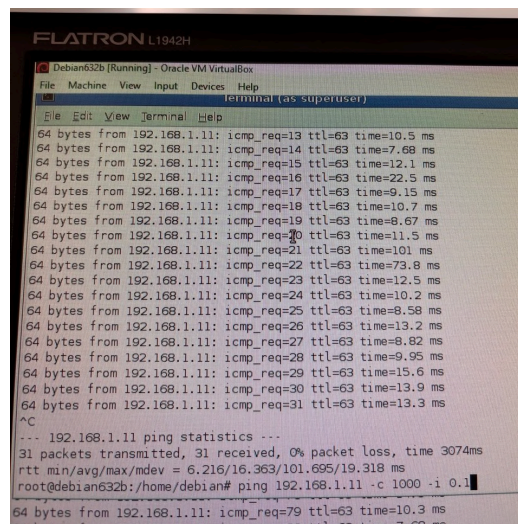
Figure 48 : AODV ping from node 15 to node 11 with RTT variation

The values are:

- Min RTT : 55.608 ms,
- Avg RTT : 70.048 ms
- Max RTT : 91.400 ms
- mDev : 11.200 ms

We do the same from node 14 as well :

Ping 192.168.1.11



```

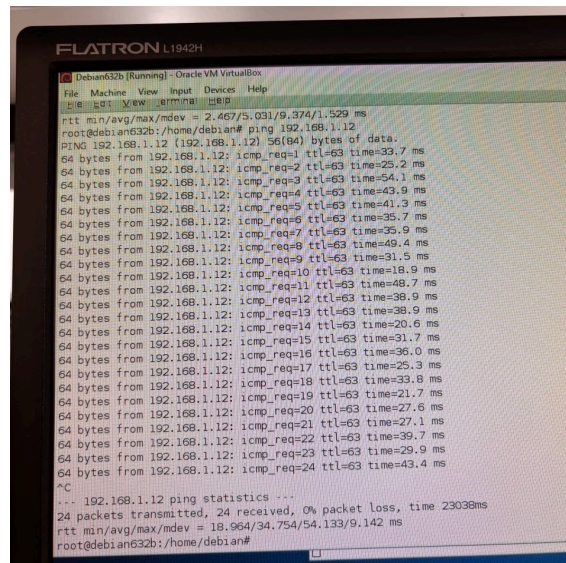
FLATRON L1942H
Debian632b [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Terminal (as superuser)
64 bytes from 192.168.1.11: icmp_req=13 ttl=63 time=10.5 ms
64 bytes from 192.168.1.11: icmp_req=14 ttl=63 time=7.68 ms
64 bytes from 192.168.1.11: icmp_req=15 ttl=63 time=12.1 ms
64 bytes from 192.168.1.11: icmp_req=16 ttl=63 time=22.5 ms
64 bytes from 192.168.1.11: icmp_req=17 ttl=63 time=9.15 ms
64 bytes from 192.168.1.11: icmp_req=18 ttl=63 time=10.7 ms
64 bytes from 192.168.1.11: icmp_req=19 ttl=63 time=8.67 ms
64 bytes from 192.168.1.11: icmp_req=20 ttl=63 time=11.5 ms
64 bytes from 192.168.1.11: icmp_req=21 ttl=63 time=101 ms
64 bytes from 192.168.1.11: icmp_req=22 ttl=63 time=73.8 ms
64 bytes from 192.168.1.11: icmp_req=23 ttl=63 time=12.5 ms
64 bytes from 192.168.1.11: icmp_req=24 ttl=63 time=10.2 ms
64 bytes from 192.168.1.11: icmp_req=25 ttl=63 time=8.58 ms
64 bytes from 192.168.1.11: icmp_req=26 ttl=63 time=13.2 ms
64 bytes from 192.168.1.11: icmp_req=27 ttl=63 time=8.82 ms
64 bytes from 192.168.1.11: icmp_req=28 ttl=63 time=9.95 ms
64 bytes from 192.168.1.11: icmp_req=29 ttl=63 time=15.6 ms
64 bytes from 192.168.1.11: icmp_req=30 ttl=63 time=13.9 ms
64 bytes from 192.168.1.11: icmp_req=31 ttl=63 time=13.3 ms
^C
--- 192.168.1.11 ping statistics ---
31 packets transmitted, 31 received, 0% packet loss, time 3074ms
rtt min/avg/max/mdev = 6.216/16.363/101.695/19.318 ms
root@debian632b: /home/debian# ping 192.168.1.11 -c 1000 -i 0.1
64 bytes from 192.168.1.11: icmp_req=79 ttl=63 time=10.3 ms
64 bytes from 192.168.1.11: icmp_req=80 ttl=63 time=7.68 ms
  
```

Figure 49 : AODV ping from node 14 to node 11 with RTT variation.

The values are :

- Min RTT : 6.216 ms
- Avg RTT : 16.363 ms
- Max RTT : 101.695 ms
- mDev : 19.318 ms

Ping 192.168.1.12



```

rtt min/avg/max/mdev = 2.467/5.031/9.374/1.529 ms
root@debian632b:/home/debian# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12) 56(84) bytes of data:
64 bytes from 192.168.1.12: icmp_req=1 ttl=63 time=33.7 ms
64 bytes from 192.168.1.12: icmp_req=2 ttl=63 time=25.2 ms
64 bytes from 192.168.1.12: icmp_req=3 ttl=63 time=54.1 ms
64 bytes from 192.168.1.12: icmp_req=4 ttl=63 time=43.9 ms
64 bytes from 192.168.1.12: icmp_req=5 ttl=63 time=41.9 ms
64 bytes from 192.168.1.12: icmp_req=6 ttl=63 time=35.7 ms
64 bytes from 192.168.1.12: icmp_req=7 ttl=63 time=35.9 ms
64 bytes from 192.168.1.12: icmp_req=8 ttl=63 time=49.4 ms
64 bytes from 192.168.1.12: icmp_req=9 ttl=63 time=31.5 ms
64 bytes from 192.168.1.12: icmp_req=10 ttl=63 time=18.9 ms
64 bytes from 192.168.1.12: icmp_req=11 ttl=63 time=48.7 ms
64 bytes from 192.168.1.12: icmp_req=12 ttl=63 time=36.9 ms
64 bytes from 192.168.1.12: icmp_req=13 ttl=63 time=36.9 ms
64 bytes from 192.168.1.12: icmp_req=14 ttl=63 time=20.6 ms
64 bytes from 192.168.1.12: icmp_req=15 ttl=63 time=31.7 ms
64 bytes from 192.168.1.12: icmp_req=16 ttl=63 time=36.0 ms
64 bytes from 192.168.1.12: icmp_req=17 ttl=63 time=25.3 ms
64 bytes from 192.168.1.12: icmp_req=18 ttl=63 time=33.8 ms
64 bytes from 192.168.1.12: icmp_req=19 ttl=63 time=21.7 ms
64 bytes from 192.168.1.12: icmp_req=20 ttl=63 time=27.6 ms
64 bytes from 192.168.1.12: icmp_req=21 ttl=63 time=27.1 ms
64 bytes from 192.168.1.12: icmp_req=22 ttl=63 time=39.7 ms
64 bytes from 192.168.1.12: icmp_req=23 ttl=63 time=29.9 ms
64 bytes from 192.168.1.12: icmp_req=24 ttl=63 time=43.4 ms
^C
... 192.168.1.12 ping statistics ...
24 packets transmitted, 24 received, 0% packet loss, time 23038ms
rtt min/avg/max/mdev = 18.964/34.754/54.133/9.142 ms
root@debian632b:/home/debian#

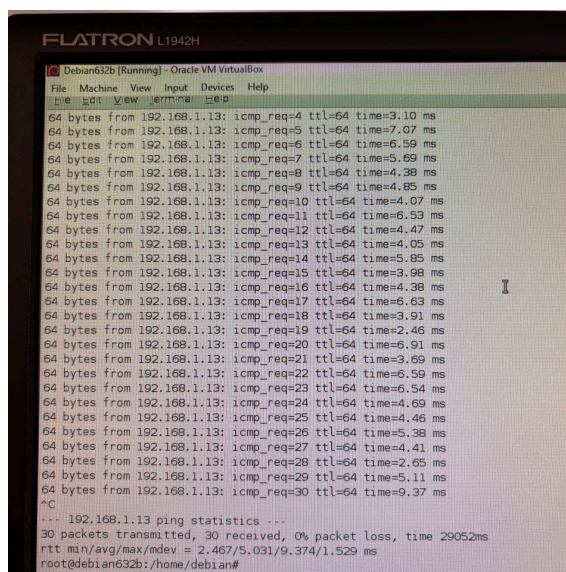
```

Figure 50 : AODV ping from node 14 to node 12 with RTT variation.

The values are :

- Min RTT : 18.964 ms
- Avg RTT : 34.754 ms
- Max RTT : 54.133 ms
- mDev : 9.142 ms

Ping 192.168.1.13



```

64 bytes from 192.168.1.13: icmp_req=4 ttl=64 time=3.10 ms
64 bytes from 192.168.1.13: icmp_req=5 ttl=64 time=7.07 ms
64 bytes from 192.168.1.13: icmp_req=6 ttl=64 time=6.59 ms
64 bytes from 192.168.1.13: icmp_req=7 ttl=64 time=5.69 ms
64 bytes from 192.168.1.13: icmp_req=8 ttl=64 time=4.38 ms
64 bytes from 192.168.1.13: icmp_req=9 ttl=64 time=4.85 ms
64 bytes from 192.168.1.13: icmp_req=10 ttl=64 time=4.07 ms
64 bytes from 192.168.1.13: icmp_req=11 ttl=64 time=6.53 ms
64 bytes from 192.168.1.13: icmp_req=12 ttl=64 time=4.47 ms
64 bytes from 192.168.1.13: icmp_req=13 ttl=64 time=4.05 ms
64 bytes from 192.168.1.13: icmp_req=14 ttl=64 time=5.85 ms
64 bytes from 192.168.1.13: icmp_req=15 ttl=64 time=3.98 ms
64 bytes from 192.168.1.13: icmp_req=16 ttl=64 time=4.38 ms
64 bytes from 192.168.1.13: icmp_req=17 ttl=64 time=6.63 ms
64 bytes from 192.168.1.13: icmp_req=18 ttl=64 time=3.91 ms
64 bytes from 192.168.1.13: icmp_req=19 ttl=64 time=2.46 ms
64 bytes from 192.168.1.13: icmp_req=20 ttl=64 time=6.91 ms
64 bytes from 192.168.1.13: icmp_req=21 ttl=64 time=3.69 ms
64 bytes from 192.168.1.13: icmp_req=22 ttl=64 time=6.59 ms
64 bytes from 192.168.1.13: icmp_req=23 ttl=64 time=6.54 ms
64 bytes from 192.168.1.13: icmp_req=24 ttl=64 time=4.69 ms
64 bytes from 192.168.1.13: icmp_req=25 ttl=64 time=4.46 ms
64 bytes from 192.168.1.13: icmp_req=26 ttl=64 time=5.38 ms
64 bytes from 192.168.1.13: icmp_req=27 ttl=64 time=4.41 ms
64 bytes from 192.168.1.13: icmp_req=28 ttl=64 time=2.65 ms
64 bytes from 192.168.1.13: icmp_req=29 ttl=64 time=5.11 ms
64 bytes from 192.168.1.13: icmp_req=30 ttl=64 time=9.37 ms
^C
... 192.168.1.13 ping statistics ...
30 packets transmitted, 30 received, 0% packet loss, time 29052ms
rtt min/avg/max/mdev = 2.467/5.031/9.374/1.529 ms
root@debian632b:/home/debian#

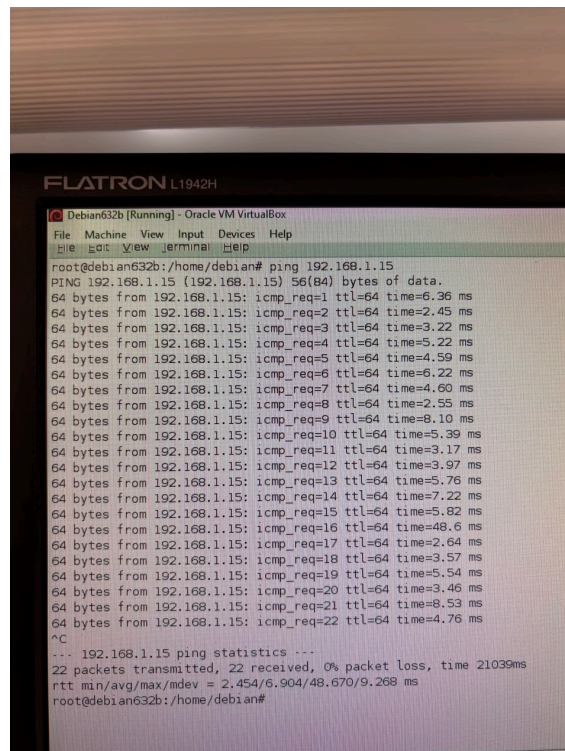
```

Figure 51 : AODV ping from node 14 to node 13 with RTT variation.

The values are :

- Min RTT : 2.467 ms
- Avg RTT : 5.031 ms
- Max RTT : 9.374 ms
- mDev : 1.529 ms

Ping 192.168.1.15

A screenshot of a terminal window titled "Debian632b [Running] - Oracle VM VirtualBox". The terminal shows the command "ping 192.168.1.15" being executed. The output displays 22 ping requests, each showing the source IP (192.168.1.15), the destination IP (192.168.1.15), the TTL (64), and the time taken in milliseconds. The times vary significantly, starting around 2.46 ms and ending around 4.76 ms. At the bottom, a summary line shows "22 packets transmitted, 22 received, 0% packet loss, time 21039ms" and "rtt min/avg/max/mdev = 2.454/6.904/48.670/9.268 ms".

```
root@debian632b:/home/debian# ping 192.168.1.15
PING 192.168.1.15 (192.168.1.15) 56(84) bytes of data.
64 bytes from 192.168.1.15: icmp_req=1 ttl=64 time=2.46 ms
64 bytes from 192.168.1.15: icmp_req=2 ttl=64 time=2.45 ms
64 bytes from 192.168.1.15: icmp_req=3 ttl=64 time=3.22 ms
64 bytes from 192.168.1.15: icmp_req=4 ttl=64 time=5.22 ms
64 bytes from 192.168.1.15: icmp_req=5 ttl=64 time=4.59 ms
64 bytes from 192.168.1.15: icmp_req=6 ttl=64 time=6.22 ms
64 bytes from 192.168.1.15: icmp_req=7 ttl=64 time=4.60 ms
64 bytes from 192.168.1.15: icmp_req=8 ttl=64 time=2.55 ms
64 bytes from 192.168.1.15: icmp_req=9 ttl=64 time=8.10 ms
64 bytes from 192.168.1.15: icmp_req=10 ttl=64 time=5.39 ms
64 bytes from 192.168.1.15: icmp_req=11 ttl=64 time=3.17 ms
64 bytes from 192.168.1.15: icmp_req=12 ttl=64 time=3.97 ms
64 bytes from 192.168.1.15: icmp_req=13 ttl=64 time=5.76 ms
64 bytes from 192.168.1.15: icmp_req=14 ttl=64 time=7.22 ms
64 bytes from 192.168.1.15: icmp_req=15 ttl=64 time=5.82 ms
64 bytes from 192.168.1.15: icmp_req=16 ttl=64 time=48.6 ms
64 bytes from 192.168.1.15: icmp_req=17 ttl=64 time=2.64 ms
64 bytes from 192.168.1.15: icmp_req=18 ttl=64 time=3.57 ms
64 bytes from 192.168.1.15: icmp_req=19 ttl=64 time=5.54 ms
64 bytes from 192.168.1.15: icmp_req=20 ttl=64 time=3.46 ms
64 bytes from 192.168.1.15: icmp_req=21 ttl=64 time=8.53 ms
64 bytes from 192.168.1.15: icmp_req=22 ttl=64 time=4.76 ms
^C
--- 192.168.1.15 ping statistics ---
22 packets transmitted, 22 received, 0% packet loss, time 21039ms
rtt min/avg/max/mdev = 2.454/6.904/48.670/9.268 ms
root@debian632b:/home/debian#
```

Figure 52: AODV ping from node 14 to node 15 with RTT variation

The values are :

- Min RTT : 2.454 ms
- Avg RTT : 6.904 ms
- Max RTT : 48.670 ms
- mDev : 9.268 ms

At the beginning of the experiment, some packets are lost due to an implementation problem related to the interaction with the ARP (Address Resolution Protocol). During the first attempts, the protocol still needs to discover and establish both the forward and backward paths between the nodes. Typically, after around three attempts, the route is correctly created and communication becomes stable. Although some important information is lost during this initialization phase, the collected data are still considered

valid for the analysis. For each set of **ping** commands, 20 RTT samples are obtained, and the minimum, average, and maximum RTT values are recorded. These values are then represented using three different curves in order to analyze the general tendency of the end-to-end delay as the number of hops increases. In general, all curves should show an increasing tendency, since packets must traverse more intermediate nodes. However, because Wi-Fi communications include several random effects, the curves do not grow perfectly equally. Wireless transmissions are affected by factors such as frame losses, retransmissions, interference, propagation conditions, and multiple retries at the link layer before a packet successfully reaches the next hop.

Among the three metrics, the minimum RTT is usually the least affected by random effects and therefore follows a more linear and theoretical tendency, with little influence from interference or retransmissions. The average RTT includes the influence of randomness and fluctuations observed during the experiment, producing a smoother curve located between the minimum and maximum values. The maximum RTT reflects the worst transmission conditions and is the most affected by delays and retransmissions. Similar observations also apply to the OLSR protocol. In the worst-case scenario, the maximum RTT measured is approximately 132 ms, which is still below the typical real-time communication threshold of 150 ms. Therefore, under this scenario, the network can still be considered suitable for real-time communications.

The experiments also show that delays are slightly greater because the nodes are running inside virtual machines, although the general behavior remains the same. During the measurements, AODV continuously exchanges Hello messages between neighboring nodes, similarly to OLSR, where Hello packets are periodically broadcast every second in order to maintain neighborhood information. However, these Hello messages do not significantly affect the RTT results. Ping packets are very short and, at a transmission rate of 54 Mbps, they occupy the wireless medium for only a few microseconds. Consequently, the channel occupation generated by both Hello and Ping packets is almost negligible, making collisions very unlikely. The observed fluctuations are therefore more likely caused by external interference and the inherent randomness of wireless communications rather than by the routing control traffic itself.

	Min RTT	Avg RTT	Max RTT	mDev rtt
Node 11	6.216 ms	16.363 ms	101.695 ms	19.318 ms
Node 12	18.964 ms	34.754 ms	54.133 ms	9.142 ms
Node 13	2.467ms	5.031 ms	9.374 ms	1.529 ms
Node 15	2.454 ms	6.904ms	48.670ms	9.268ms

The summary of the RTT variation from different nodes from node 14 is as follows :

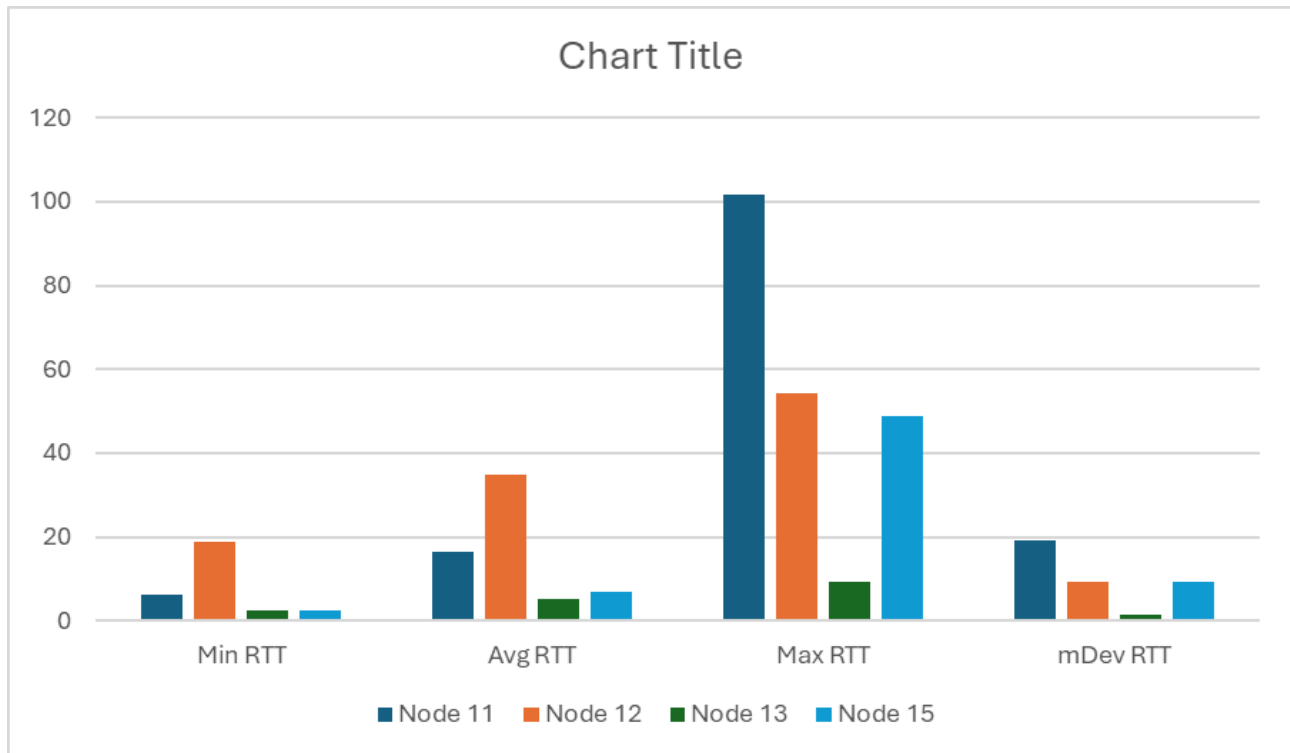


Figure 53 : AODV RTT measurements from node 14

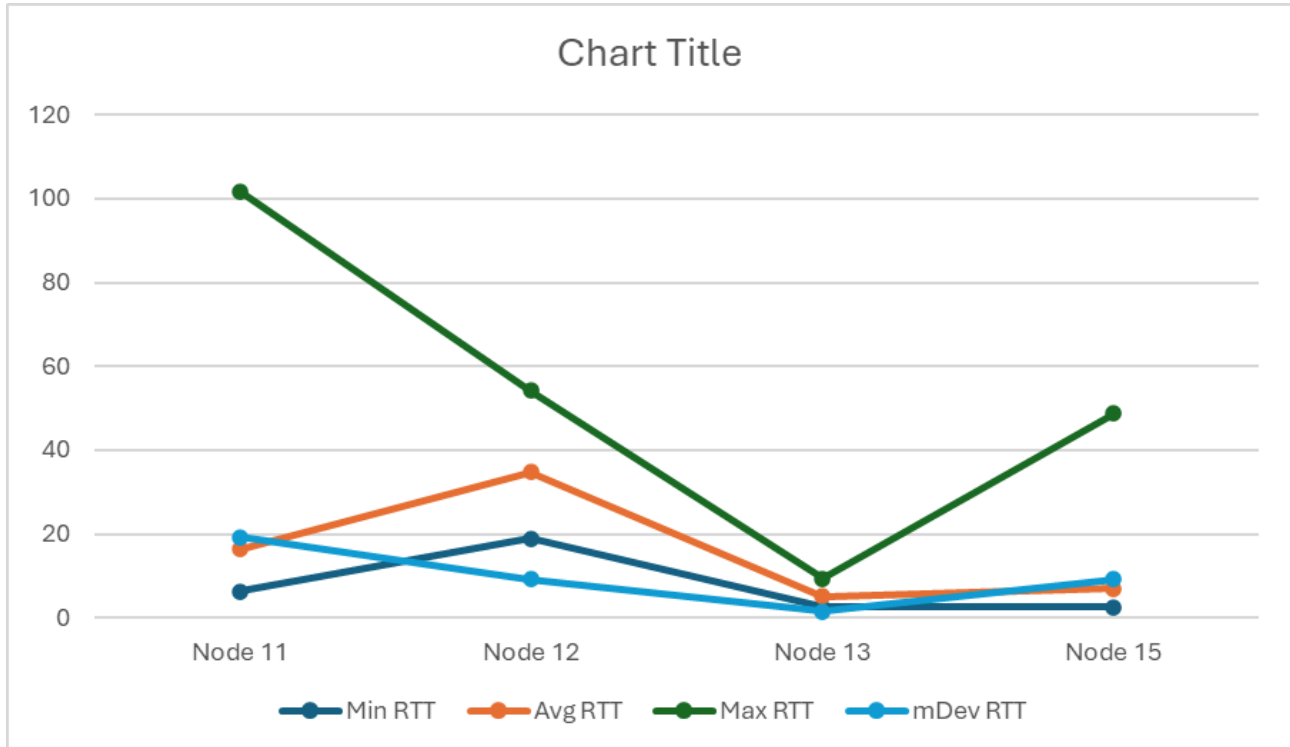


Figure 54 : AODV RTT metrics from node 14

And the supportive information from node 15 is as bellow :

	Min RTT	Avg RTT	Max RTT	mDev rtt
Node 11	55.608 ms	70.048 ms	91.400 ms	11.200 ms
Node 12	26.635 ms	41.445 ms	65.437 ms	9.910 ms
Node 13	6.830 ms	12.652 ms	32.036 ms	5.346 ms
Node 14	2.146 ms	6.050 ms	14.863 ms	2.698 ms

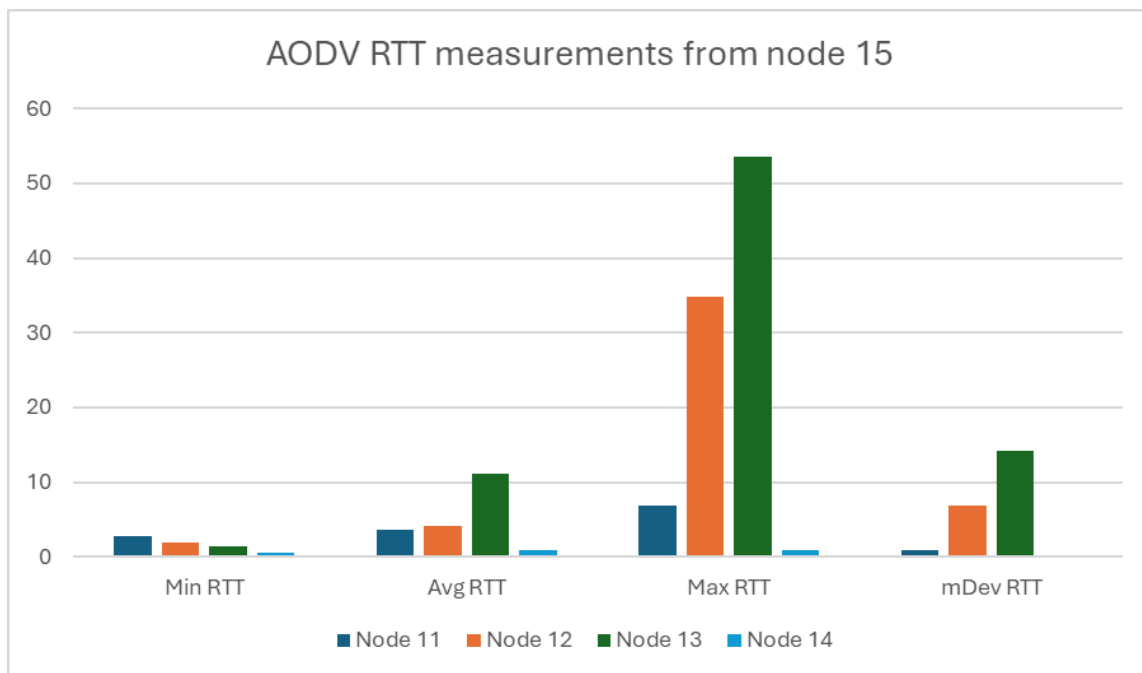


Figure 55 : AODV RTT measurements from node 15

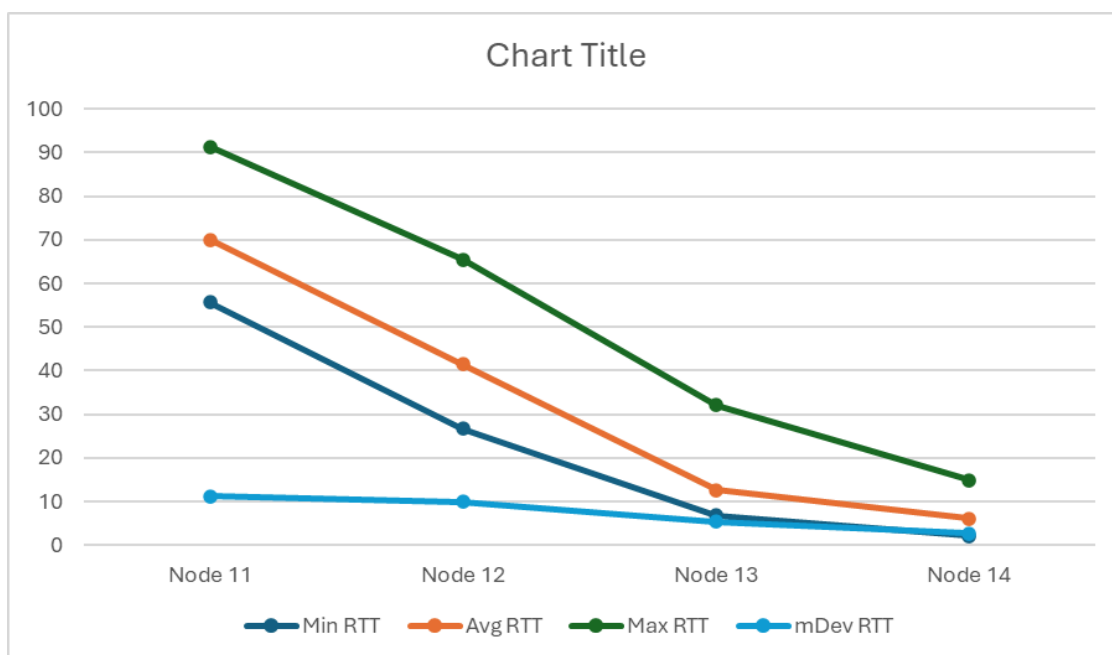


Figure 56 : AODV RTT metrics from node 15

After modifying the topology, the node can now communicate directly with nodes 11 and 15. In this experiment, the objective is to measure the **route change latency**, which is the time required for the network to discover a new route after a link failure occurs.

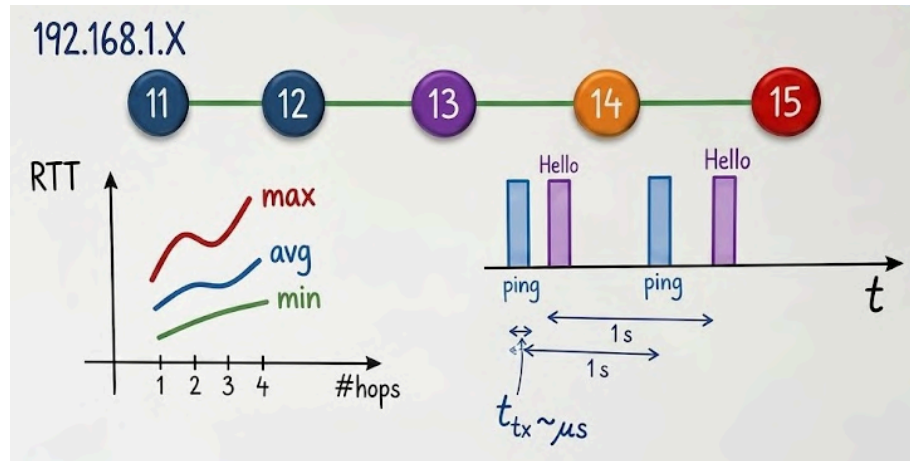


Figure 57: RTT as a function of number of hops and temporal relationship between Ping and Hello messages.

Initially, there are two possible paths between the source and destination nodes. Although the routing protocol may sometimes choose the upper path, the preferred route should be the lower one according to the configured topology. To force a route change, node 15 is disabled, generating a link failure in the network. As a consequence, node 11 stops receiving Hello messages from node 15, allowing AODV to detect that the neighbor is no longer reachable. At that moment, a new route discovery process is started in order to find an alternative path to the destination. The time elapsed between the link failure and the establishment of the new valid route is known as the route change latency. During this process, the routing table is updated and the new neighbor relationship with 192.168.1.11 can be observed once connectivity is restored through the alternative route.

New neighbor 192.168.1.11.

The measured route change latency results were not directly obtained during the experiment and were instead provided by the professor. Theoretical and experimental values can differ because of implementation details and the execution environment.

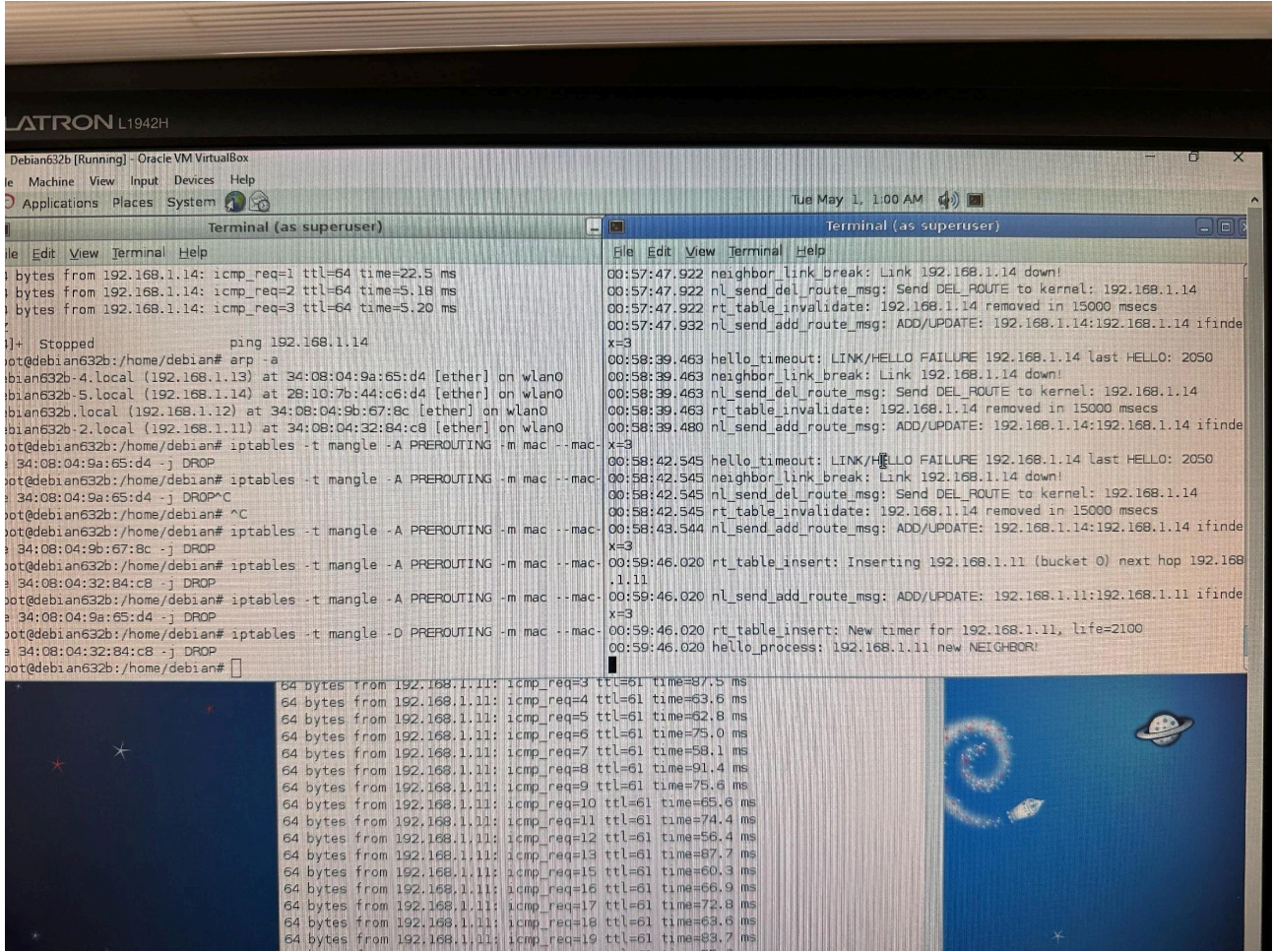


Figure 58: AODV route recovery process after a link failure and neighbor rediscovery

In AODV, route change latency corresponds to the time required to detect a link failure and discover a new valid route. This latency can be expressed as:

$$RCL = T_{det.linkfailure} + T_{route\ discovery}$$

where $T_{det.linkfailure}$ is the time needed to detect that a neighboring node has disappeared, and $T_{route\ discovery}$ is the time required to establish a new route, which is approximately equivalent to a round-trip time. In AODV, Hello messages are transmitted periodically every 1 second. A node is considered disconnected after two consecutive Hello messages are lost. Therefore, the link failure detection time is a random value between one and two Hello intervals, depending on whether the failure occurs immediately before or immediately after the transmission of a Hello packet. As a result, the route change latency in AODV is expected to range approximately between 1 and 2 seconds, plus the additional route discovery time.

The value obtained experimentally by the professor was approximately 2.7 seconds. This larger delay is mainly caused by the fact that the routing protocol implementation runs in user space as a daemon application instead of being implemented directly inside the kernel. Kernel implementations provide lower latency because communication between the routing protocol and the operating system is more direct and efficient. In contrast, user-space implementations introduce additional communication and processing delays between the application and the kernel. Furthermore, the use of virtual machines also increases the measured latency.

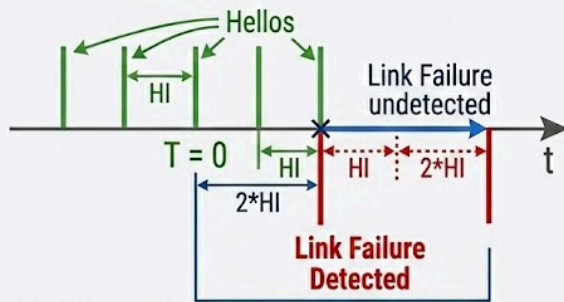
In principle, reducing the Hello interval would also reduce the route change latency because link failures would be detected faster. However, transmitting Hello messages too frequently would increase network overhead and occupy the wireless medium with control traffic, leaving less bandwidth available for data transmissions. In OLSR, for example, the Hello interval is typically 2 seconds and link failures are detected after the loss of 3 consecutive Hello messages. In practice, wireless experiments are highly sensitive to environmental conditions and often produce unstable results. During the laboratory sessions, interference caused by people moving nearby and other wireless devices generated fluctuations and unexpected behavior. This explains why the professor obtained results closer to the theoretical values when performing the experiments alone in a controlled environment. Wireless communications are very sensitive to interference, and the loss of consecutive Hello messages may sometimes produce irregular or unexpected routing behavior.

CONCLUSIÓN

Based on the results obtained in the project, we decided to use AODV as the routing protocol for the wireless multihop network. AODV is more suitable for this scenario because it can quickly adapt to topology changes and link failures, which are very common in disaster or rescue situations. When a node disappears, AODV is able to dynamically discover a new route and restore communication in a short time. In our experiments, the measured route change latency for AODV was approximately 2.7 seconds, while the recovery time obtained with OLSR was around 7.2 seconds, showing that AODV reacts faster to link failures.

The experiments also showed that the network can still provide acceptable performance for real-time applications such as voice or video communications. Even in the worst case, the maximum RTT measured was around 132 ms, which is still below the typical 150 ms threshold required for real-time communications. In addition, the RTT results clearly showed that the delay increases as the number of hops grows. For example, the throughput decreased from approximately 23 Mbps in the one-hop scenario to around 12 Mbps for two hops, 7 Mbps for three hops, and approximately 5–6 Mbps for four hops, mainly because of interference, retransmissions, and the half-duplex nature of Wi-Fi communications.

RCL AODV HI = HELLO_INTERVAL



$$RCL = T_{\text{Det Link Failure}} + T_{\text{Route Discovery}}$$

$$RCL = T_{\text{Det Link Failure}} + T_{\text{Discovery}}$$

$$RCL \simeq T_{\text{Det Link Failure}} \in [\text{HI}, 2*HI]$$

BY DEFAULT, IN AODV

- HELLO_INTERVAL (HI) = 1 s
- $RCL \in [1, 2] \text{ s}$ ←

EXPERIMENTAL RESULTS

↳ $RCL = 2.7 \text{ s}$

perspective

Figure 59 : AODV changing path calculation

Another important observation is that wireless communications are strongly affected by random effects such as interference, retransmissions, and medium contention. This explains why the maximum RTT values presented strong fluctuations, while the minimum RTT values followed a more linear and theoretical behavior. Despite these limitations, the experiments demonstrated that AODV offers a good balance between flexibility, fast route recovery, and communication performance. Therefore, considering the obtained results and the requirements of a rescue-team communication network, AODV is the most appropriate routing protocol for this project.